



Metaheuristic optimization

بهینه سازی فرا ابتکاری

Mohammad Khalooei

Spring 2026 — Amirkabir University of Technology





- **Mohammad Khalooei**

- PhD in Artificial Intelligence
- Lecturer at Sharif & Amirkabir university of technology



Khalooei@aut.ac.ir

mkhalooei90@gmail.com

@mkhalooei



هدف کلی

هدف این درس آشنایی و تولید الگوریتم‌های فراابتکاری برای حل مسایل مختلف بهینه‌سازی است. در این درس ابتدا الگوریتم‌های تکاملی معرفی می‌شوند و سپس تعدادی از الگوریتم‌های فراابتکاری شناخته شده معرفی می‌شوند و در نهایت ابزارهای لازم برای تولید انواع الگوریتم‌های فراابتکاری بررسی می‌شوند.

سرفصل‌ها

- تئوری ماکروسکوپی و میکروسکوپی تکامل
- الگوریتم‌های تکاملی: چارچوب کلی، عملگرهای تنوع، عملگرهای انتخاب
- الگوریتم‌های تکاملی: الگوریتم ژنتیکی، استراتژی تکاملی، برنامه‌نویسی ژنتیکی، الگوریتم ممتیک
- الگوریتم‌های جستجوی محلی: الگوریتم تپه‌نوردی، قدم زدن تصادفی، گرم و سرد کردن تدریجی
- الگوریتم‌های مبتنی بر جمعیت: گروه ذرات، کلونی مورچگان
- مدیریت محدودیت‌ها در بهینه‌سازی در الگوریتم‌های فراابتکاری
- مسائل بهینه‌سازی چند قلمای در الگوریتم‌های فراابتکاری
- مسائل بهینه‌سازی چند هدفی در الگوریتم‌های فراابتکاری
- روش‌های بهینه‌سازی پارامترهای الگوریتم‌های فراابتکاری

روش ارزشیابی (پیشنهادی)

- تمرین‌ها و پروژه: ۲۰ درصد نمره
- آزمون‌های میان‌ترم و پایانی: ۶۰ درصد نمره
- پروژه پژوهشی: ۲۰ درصد نمره

منابع علمی پیشنهادی

- [1] A. E. Eiben and J. E. Smith. *Introduction to Evolutionary Computing*. Springer, 2015.
- [2] K. A. D. Jong. *Evolutionary Computation: A Unified Approach*. MIT press, 2006.



عنوان درس به فارسی: بهینه‌سازی فرا ابتکاری		
عنوان درس به انگلیسی: Meta-Heuristic Optimization	نوع درس و واحد	
دروس پیش‌نیاز:	— پایه ☐ نظری ☒	
دروس هم‌نیاز:	— تخصصی الزامی ☒ عملی ☐	
تعداد واحد:	۳	تخصصی اختیاری ☐ نظری-عملی ☐
تعداد ساعت:	۴۸	مهارتی ☐

- **Lesson Objective:**

- This lesson aims to introduce various metaheuristic algorithms for solving different optimization problems. At the end of the lesson, students will become familiar with several well-known metaheuristic algorithms and will learn about the types of metaheuristics required for different optimization problems.

Main Topics:

- Introduction to Macro and Micro Evolutionary Computation
- Global, local, and population-based metaheuristics
- Single-objective and multi-objective metaheuristics
- Trajectory-based metaheuristics (such as Simulated Annealing, Tabu Search, etc.)
- Population-based metaheuristics (such as Genetic Algorithms, Particle Swarm Optimization, etc.)
- Constraint handling techniques in metaheuristic algorithms
- Several issues in improving the performance of metaheuristics
- Several target-oriented improvement issues in metaheuristics
- Methods for parameter tuning and adaptation in metaheuristic algorithms



- Ordinary least square

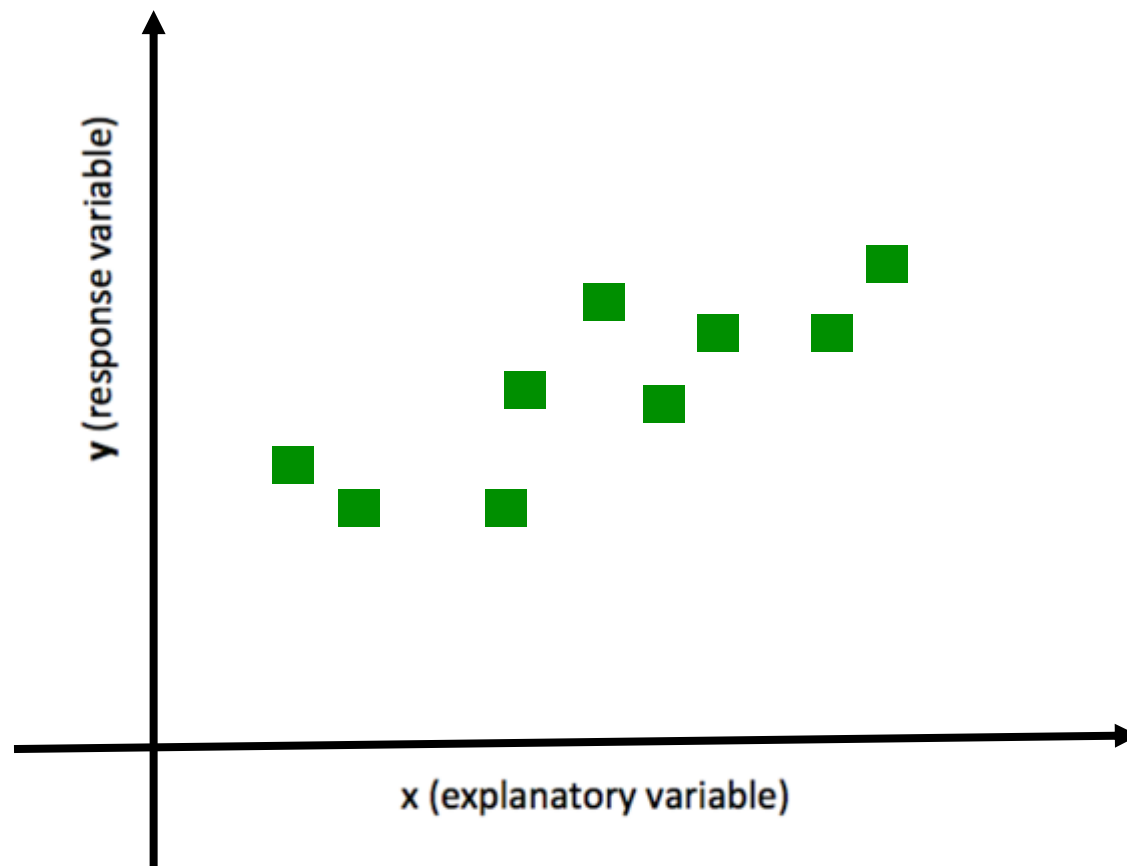
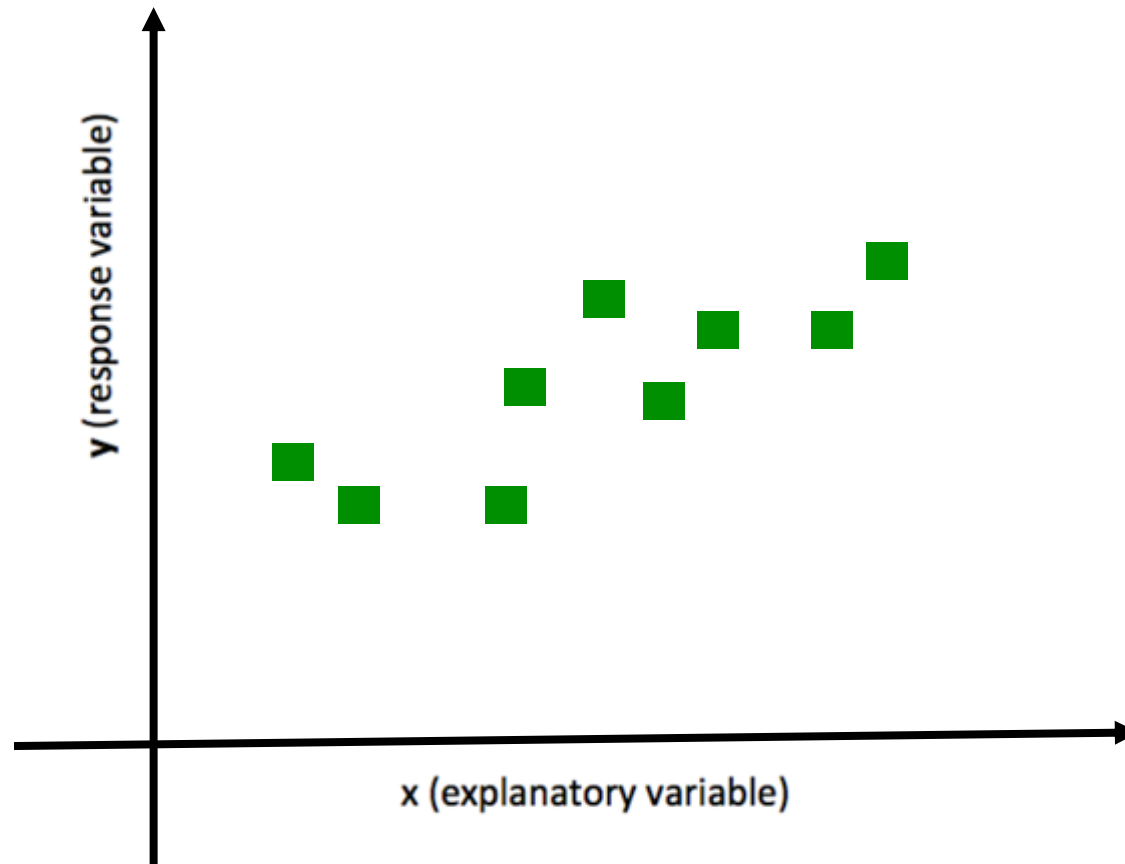


Illustration of a simple linear regression model



- Ordinary least square

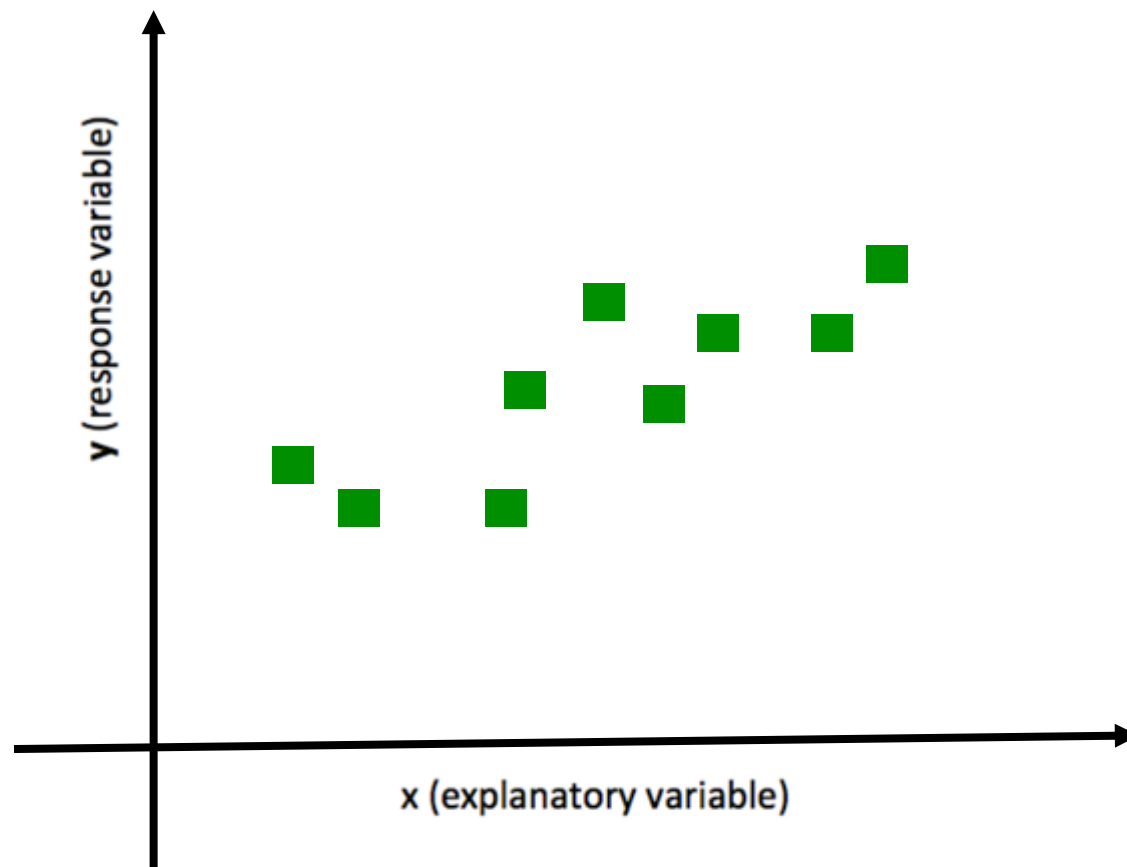


our goal is to find the line (or hyperplane) that minimizes the vertical **offsets**.

Illustration of a simple linear regression model



- Ordinary least square



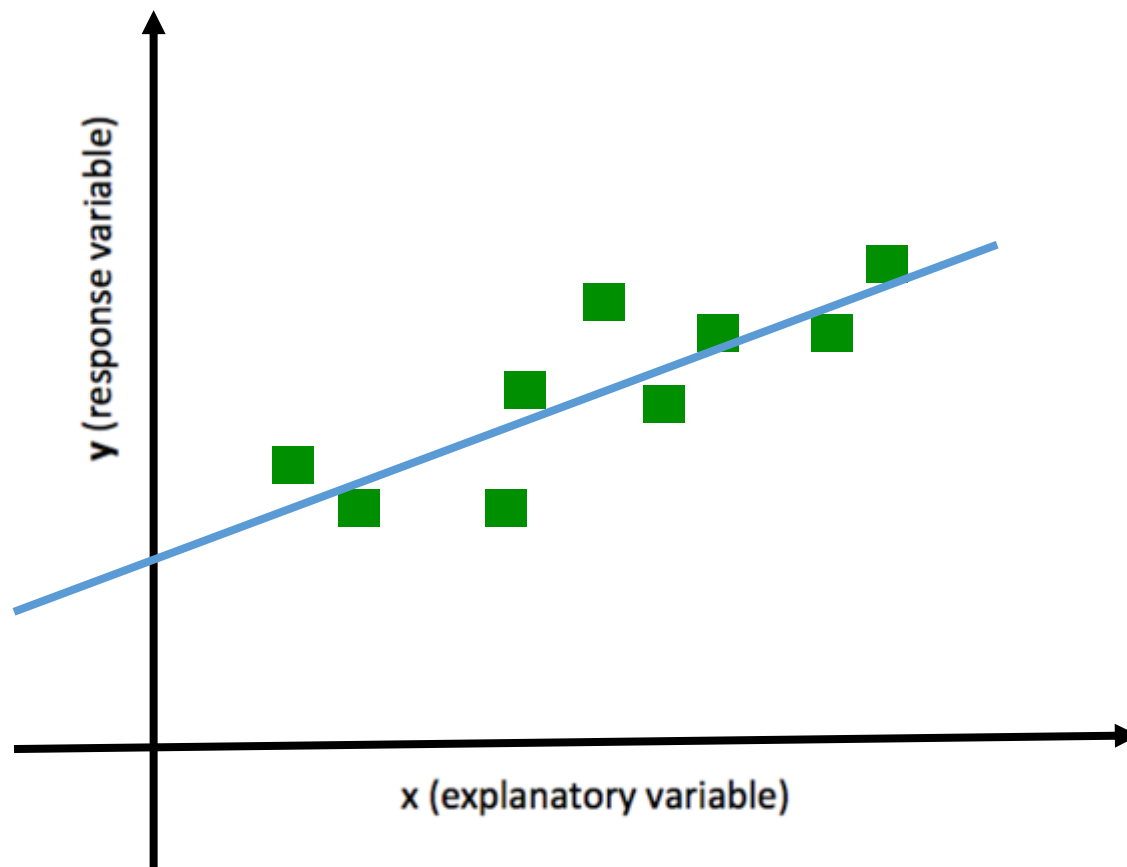
our goal is to find the line (or hyperplane) that minimizes the vertical **offsets**.

we define the **best-fitting** line as the line that minimizes the sum of *squared errors* (*SSE*) or *mean squared error* (*MSE*) between our *target variable* (*y*) and our *predicted output* over all samples in our dataset of size *n*

Illustration of a simple linear regression model



- Ordinary least square



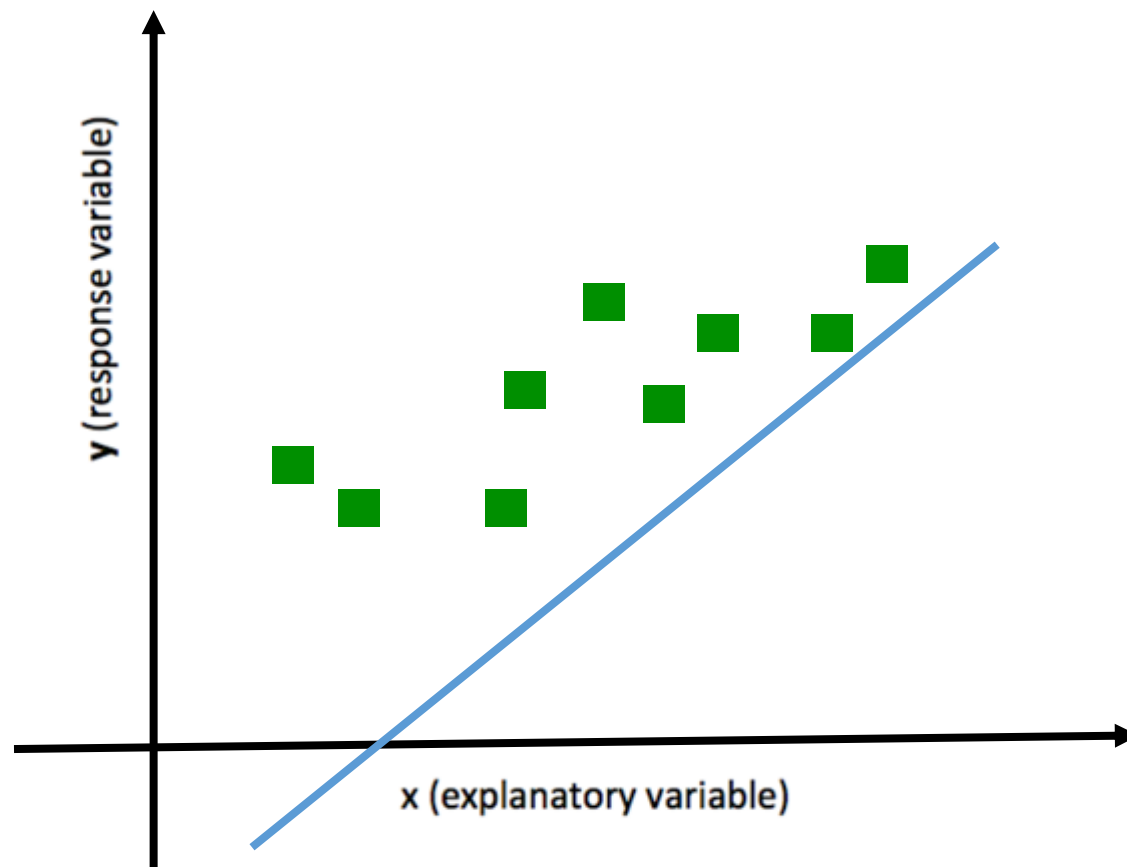
our goal is to find the line (or hyperplane) that minimizes the vertical **offsets**.

we define the **best-fitting** line as the line that minimizes the sum of *squared errors (SSE)* or *mean squared error (MSE)* between our *target variable (y)* and our *predicted output* over all samples in our dataset of size n

Illustration of a simple linear regression model



- Ordinary least square



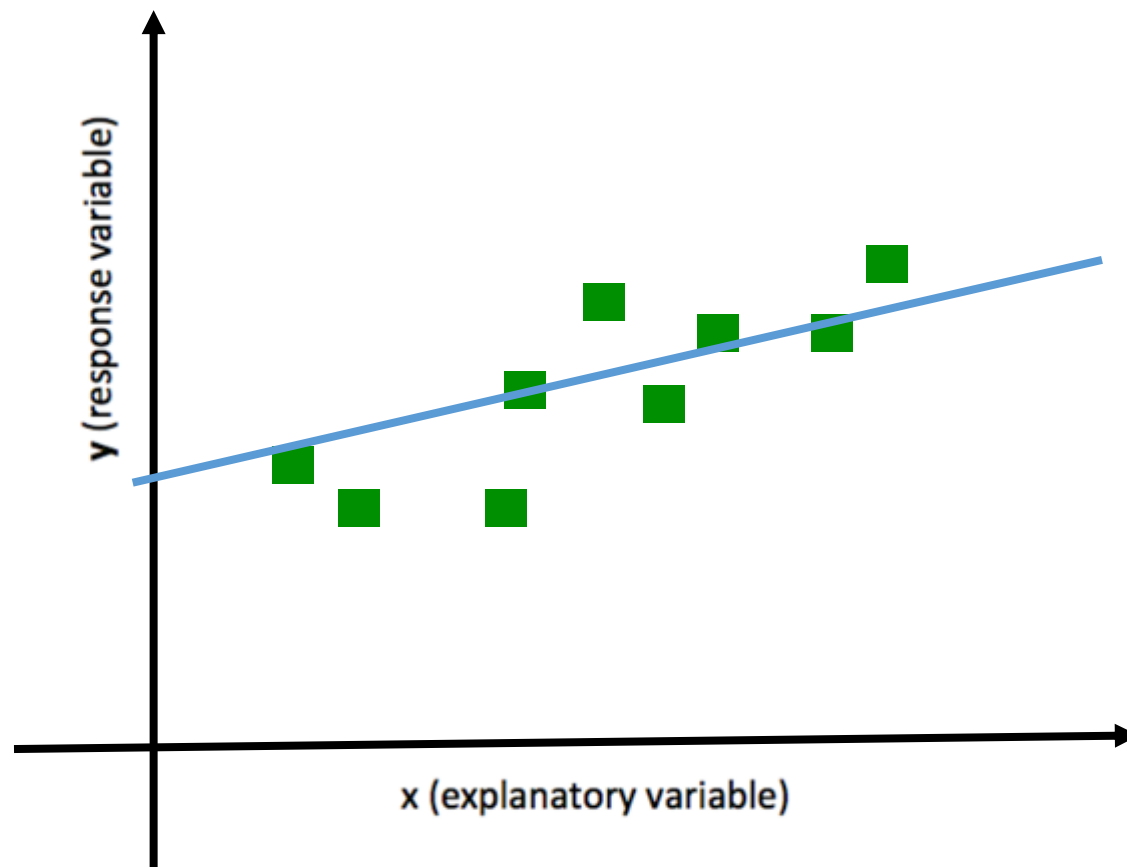
our goal is to find the line (or hyperplane) that minimizes the vertical **offsets**.

we define the **best-fitting** line as the line that minimizes the sum of *squared errors* (*SSE*) or *mean squared error* (*MSE*) between our *target variable* (*y*) and our *predicted output* over all samples in our dataset of size *n*

Illustration of a simple linear regression model



- Ordinary least square



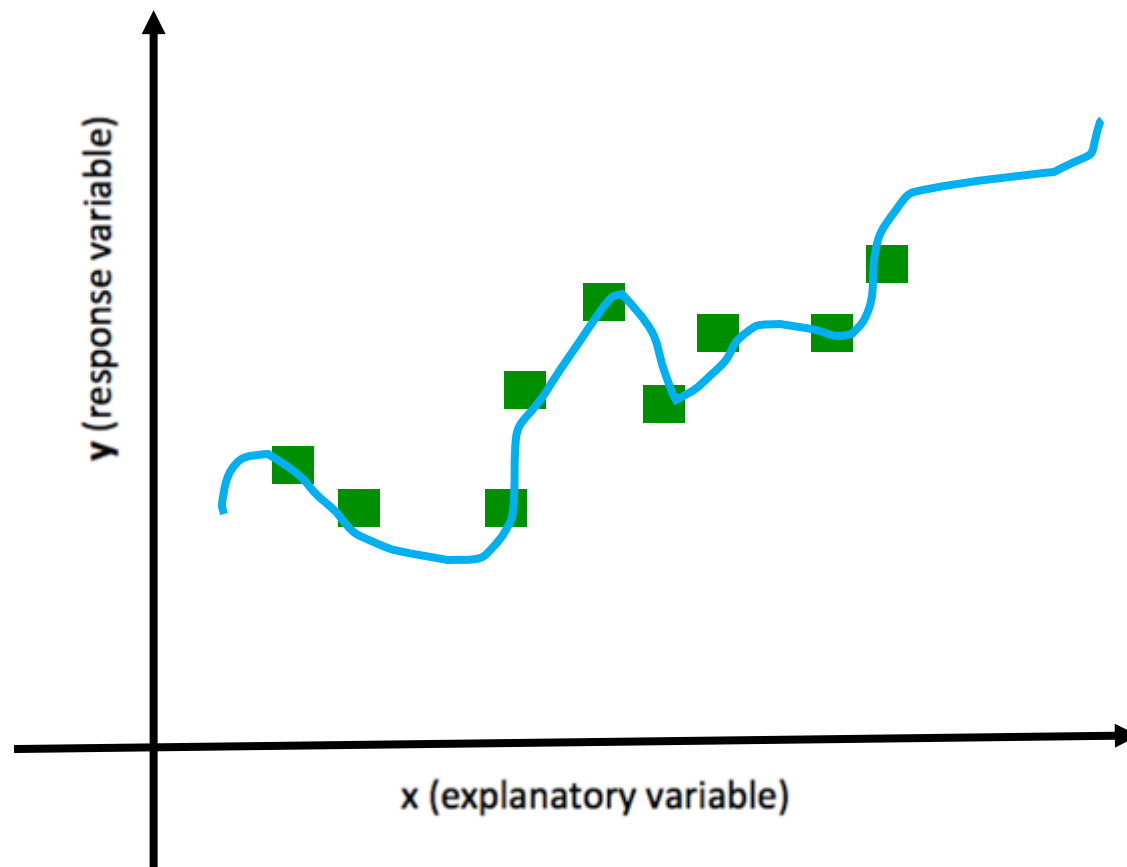
our goal is to find the line (or hyperplane) that minimizes the vertical **offsets**.

we define the **best-fitting** line as the line that minimizes the sum of *squared errors (SSE)* or *mean squared error (MSE)* between our *target variable (y)* and our *predicted output* over all samples in our dataset of size n

Illustration of a simple linear regression model



- Ordinary least square



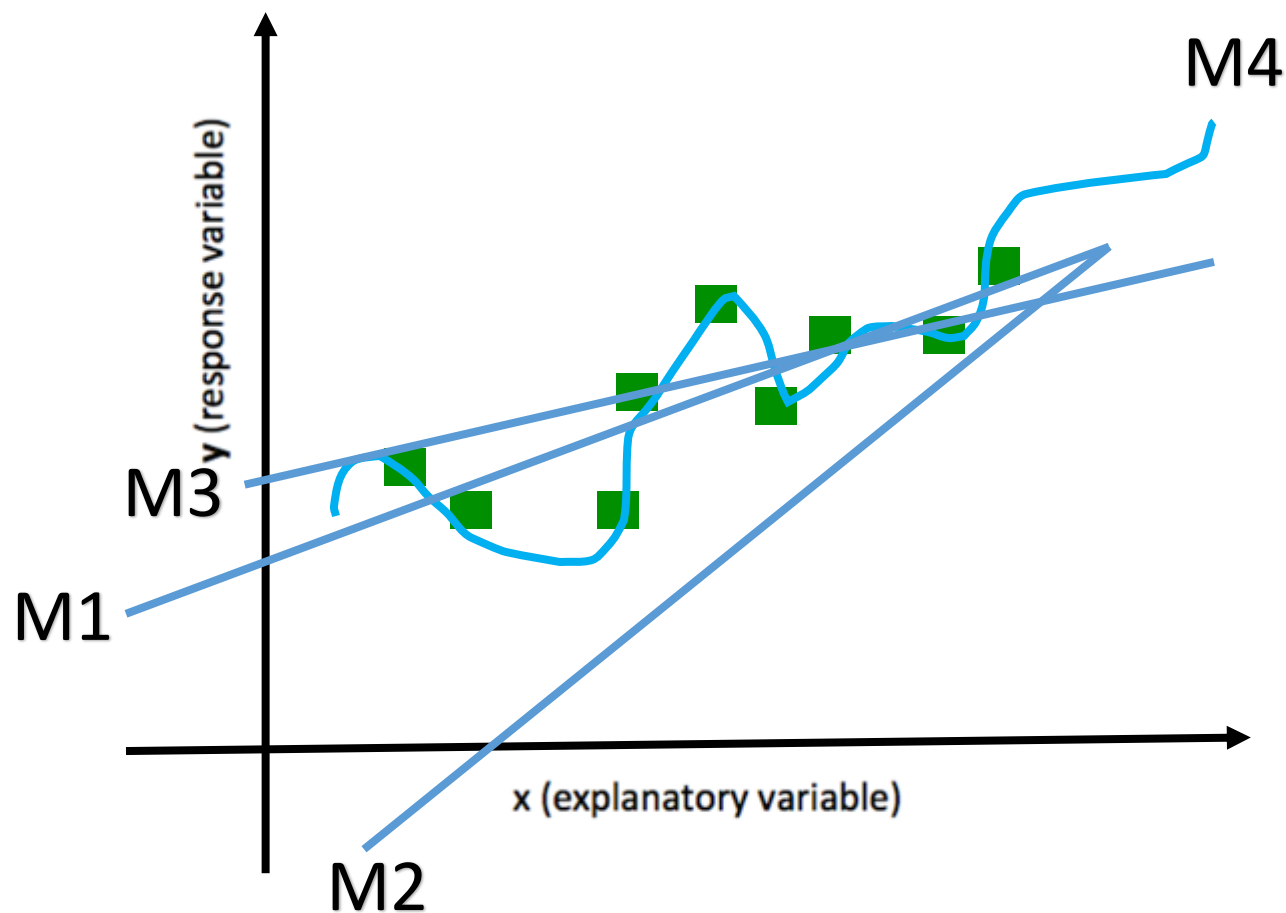
our goal is to find the line (or hyperplane) that minimizes the vertical **offsets**.

we define the **best-fitting** line as the line that minimizes the sum of *squared errors (SSE)* or *mean squared error (MSE)* between our *target variable (y)* and our *predicted output* over all samples in our dataset of size n

Illustration of a simple linear regression model



- Ordinary least square



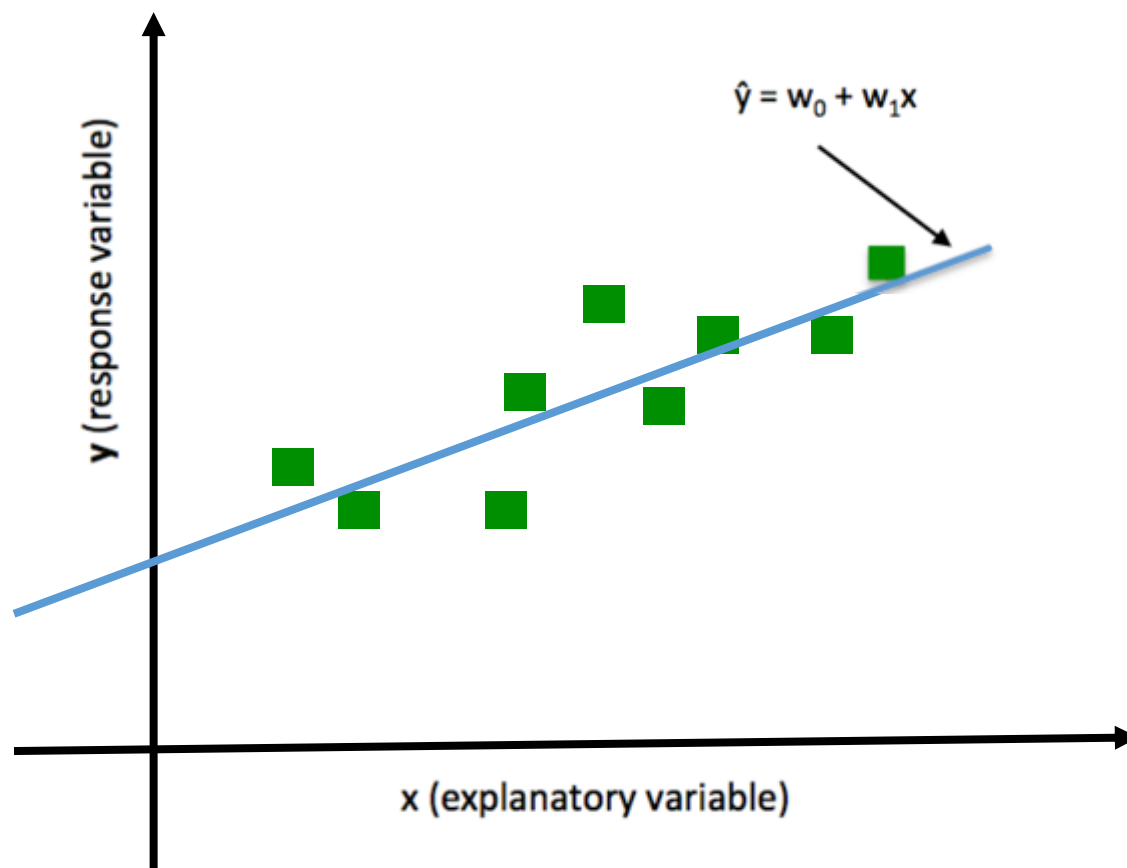
our goal is to find the line (or hyperplane) that minimizes the vertical **offsets**.

we define the **best-fitting** line as the line that minimizes the sum of *squared errors (SSE)* or *mean squared error (MSE)* between our *target variable (y)* and our *predicted output* over all samples in our dataset of size n

Illustration of a simple linear regression model



- Ordinary least square



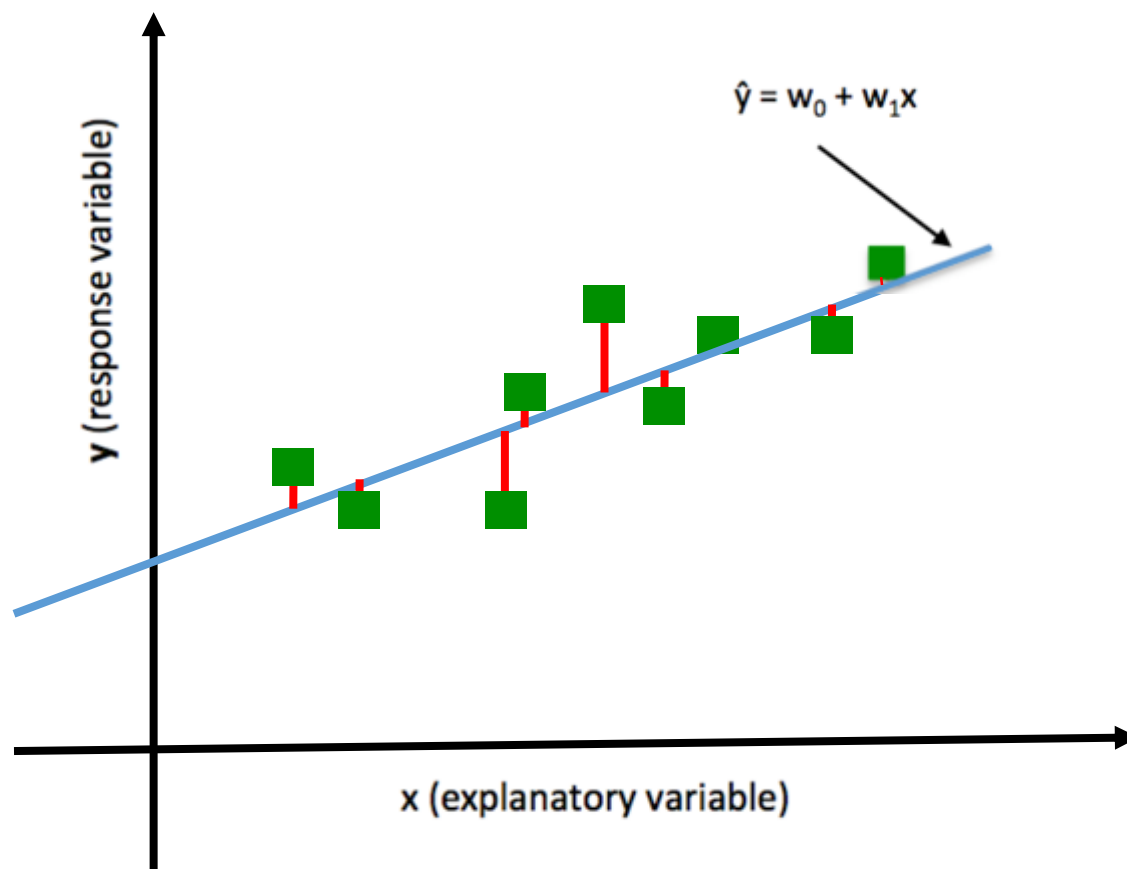
our goal is to find the line (or hyperplane) that minimizes the vertical **offsets**.

we define the **best-fitting** line as the line that minimizes the sum of *squared errors* (SSE) or *mean squared error* (MSE) between our *target variable* (y) and our *predicted output* over all samples in our dataset of size n

Illustration of a simple linear regression model



- Ordinary least square



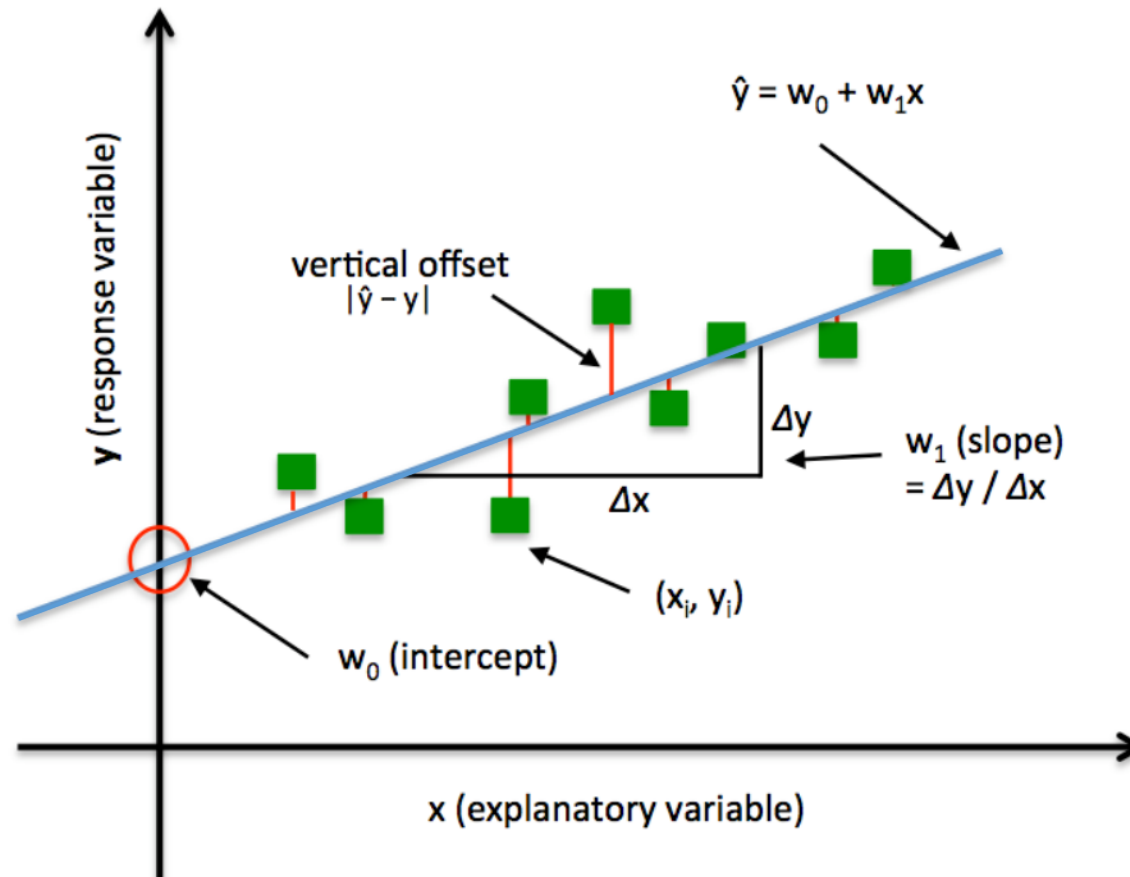
our goal is to find the line (or hyperplane) that minimizes the vertical **offsets**.

we define the **best-fitting** line as the line that minimizes the sum of *squared errors (SSE)* or *mean squared error (MSE)* between our *target variable (y)* and our *predicted output* over all samples in our dataset of size n

Illustration of a simple linear regression model



- Ordinary least square (OLS) linear regression



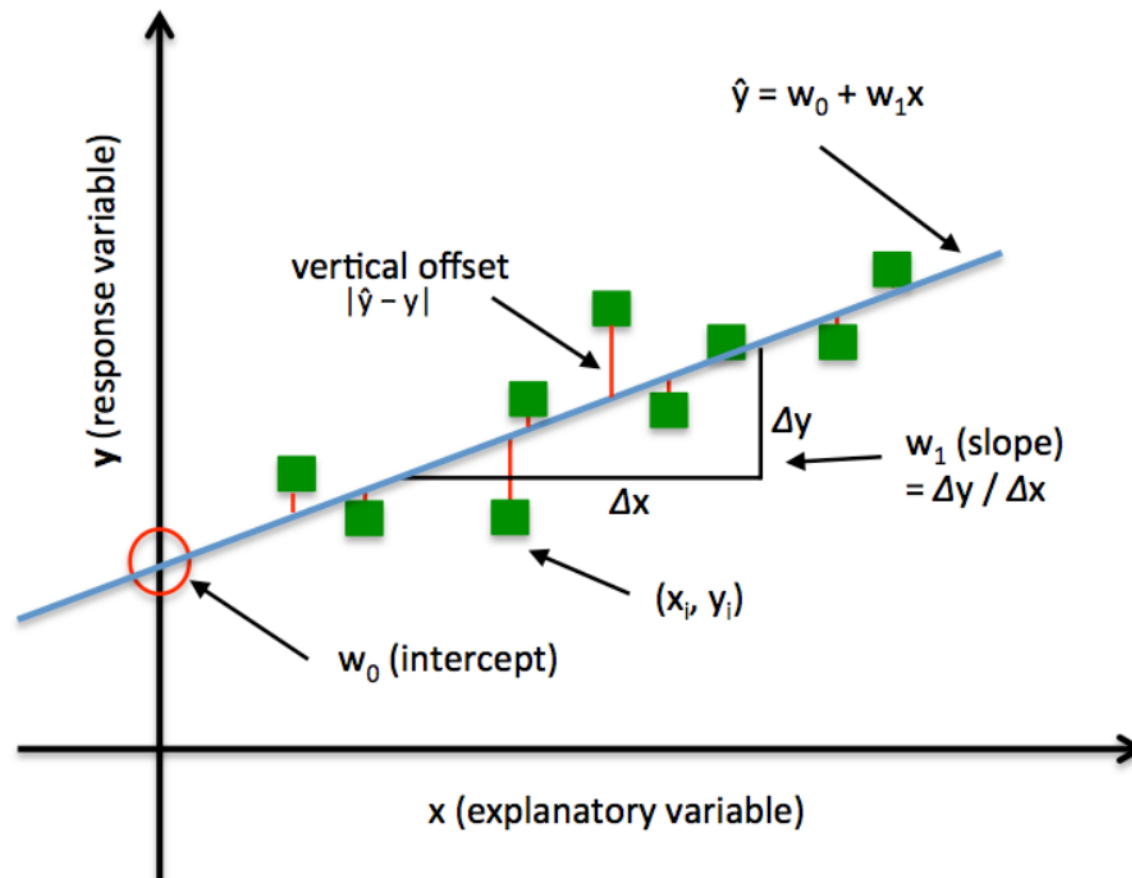
our goal is to find the line (or hyperplane) that minimizes the vertical **offsets**.

we define the **best-fitting** line as the line that minimizes the sum of *squared errors* (SSE) or *mean squared error* (MSE) between our *target variable* (y) and our *predicted output* over all samples in our dataset of size n

Illustration of a simple linear regression model



- Ordinary least square (OLS) linear regression



our goal is to find the line (or hyperplane) that minimizes the vertical **offsets**.

we define the **best-fitting** line as the line that minimizes the sum of *squared errors* (SSE) or *mean squared error* (MSE) between our *target variable* (y) and our *predicted output* over all samples in our dataset of size n

Illustration of a simple linear regression model



- Ordinary least square (OLS) linear regression

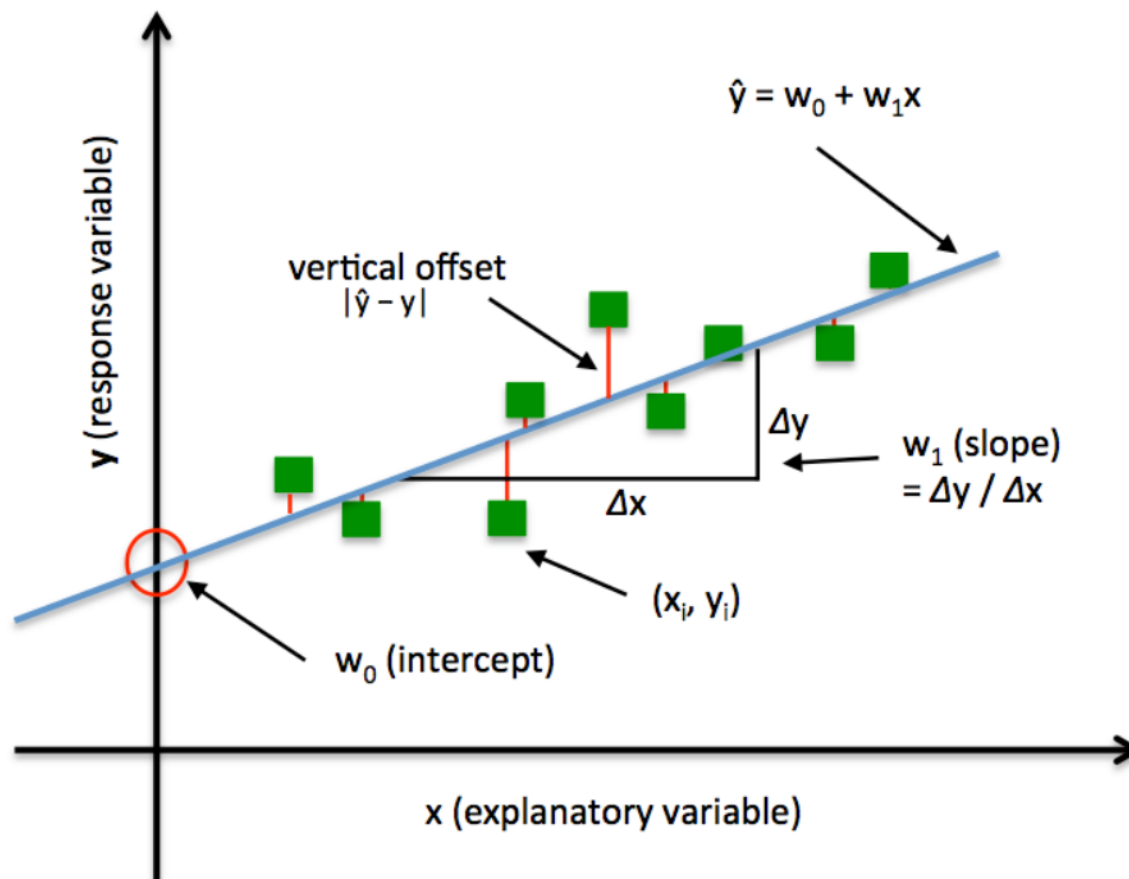


Illustration of a simple linear regression model

our goal is to find the line (or hyperplane) that minimizes the vertical **offsets**.

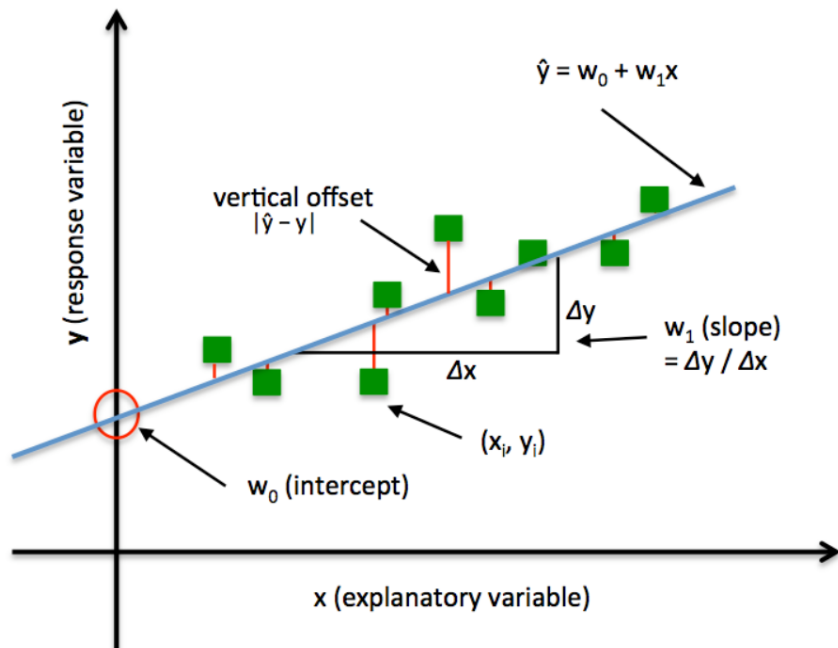
we define the **best-fitting** line as the line that minimizes the sum of *squared errors* (SSE) or *mean squared error* (MSE) between our *target variable* (y) and our *predicted output* over all samples in our dataset of size n

$$SSE = \sum_i (\text{target}^{(i)} - \text{output}^{(i)})^2$$

$$MSE = \frac{1}{n} \times SSE$$



- Ordinary least square (OLS) linear regression



The **linear function (linear regression model)** is defined as

$$y = w_0x_0 + w_1x_1 + \dots + w_mx_m = \sum_{i=0}^m = \mathbf{w}^T \mathbf{x}$$

y is the response variable

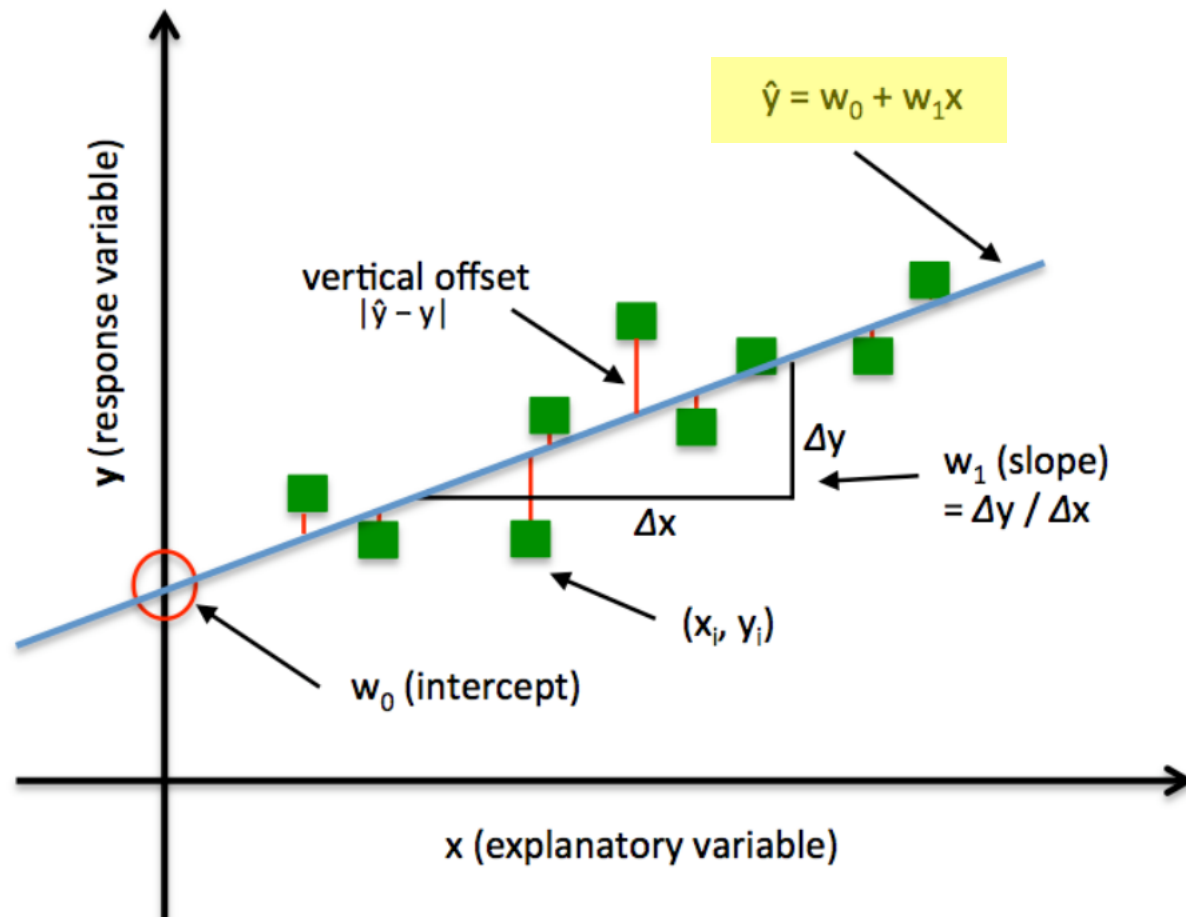
$\mathbf{X}$ is an m -dimensional sample vector

$\mathbf{w}$ is the weight vector (vector of coefficients)

Note that w_0 represents the y -axis intercept of the model and therefore $x_0=1$



- Ordinary least square (OLS) linear regression

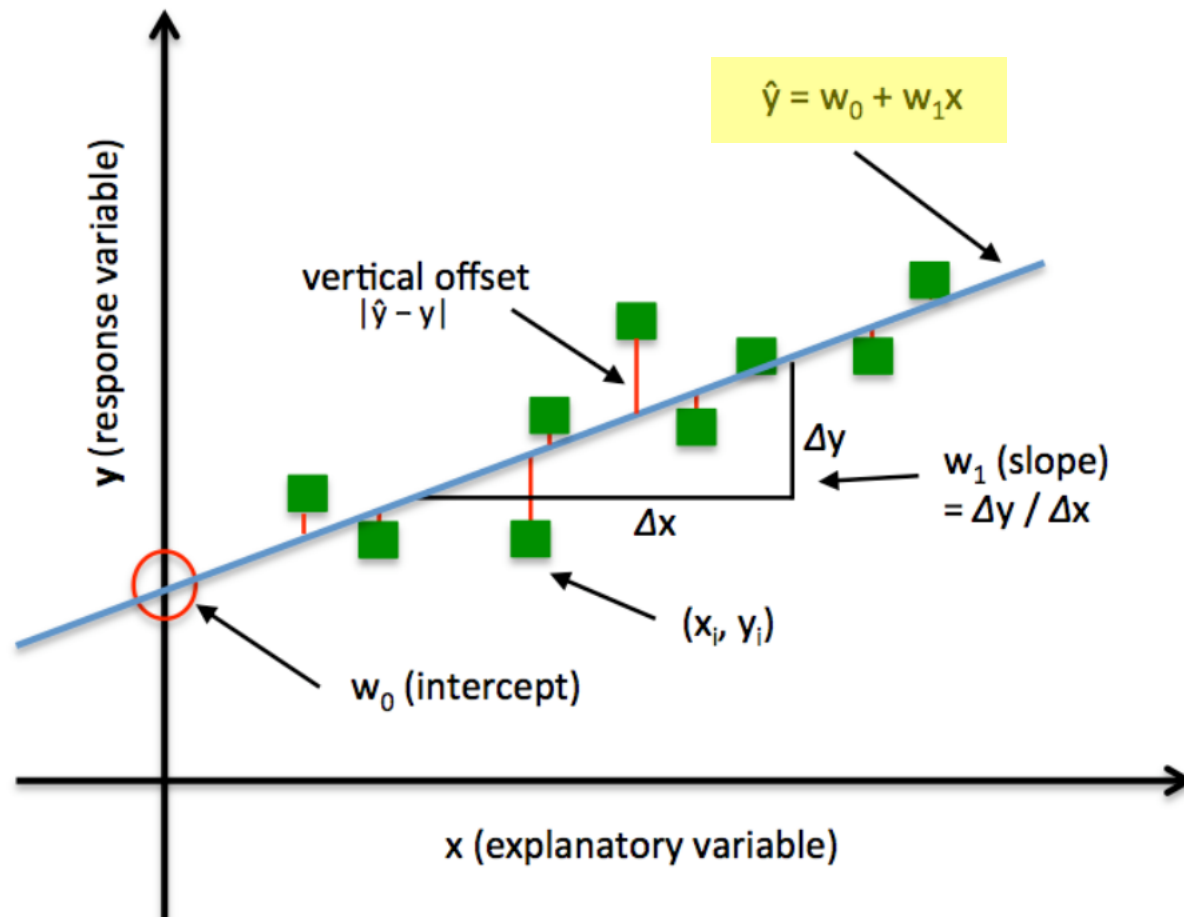


Training steps:

- 1- define model and parameters
- 2- tune parameters of model on dataset



- Ordinary least square (OLS) linear regression



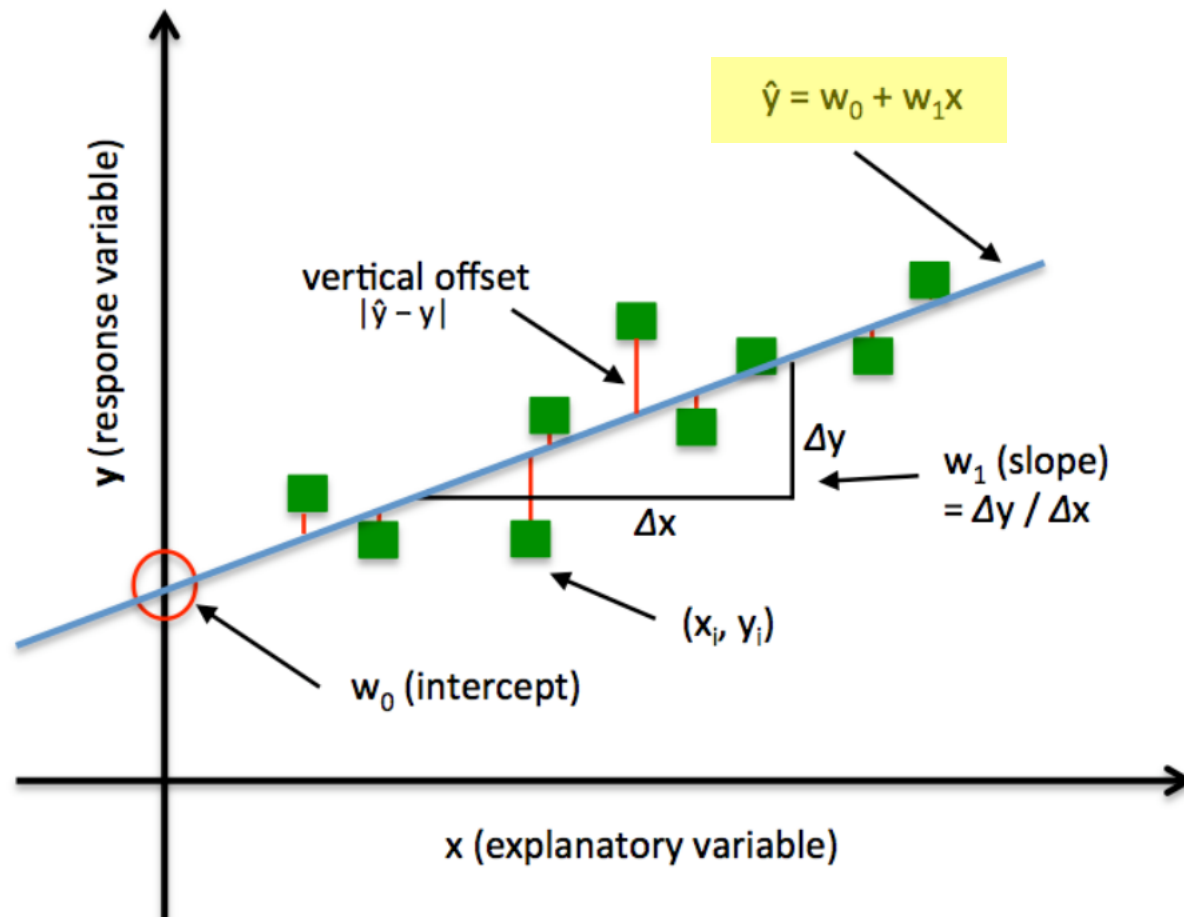
Training steps:

- 1- define model and parameters
- 2- tune parameters of model on dataset

What are the parameters?



- Ordinary least square (OLS) linear regression



Training steps:

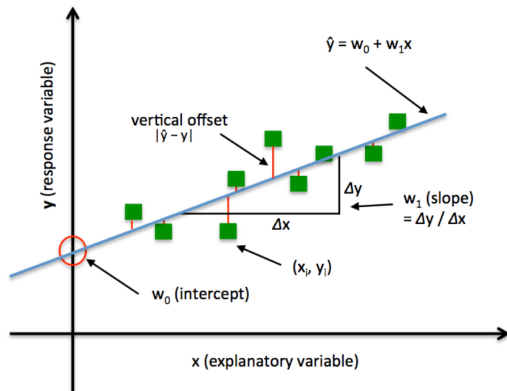
- 1- define model and parameters
- 2- tune parameters of model on dataset

What are the parameters?

w_0 and w_1



- Ordinary least square (OLS) linear regression



What are the parameters?

w_0 and w_1

$$\hat{y} = w_0 + w_1x$$

Training steps:

- 1- define model and parameters
- 2- tune parameters of model on dataset

w_0 is the intercept (where the line crosses the y-axis when $x = 0$).

w_1 is the slope (how much y changes when x increases by 1 unit).



Introduction



```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
```

```
# ----- Example Dataset -----
```

```
np.random.seed(42)
X = np.random.rand(20) * 10          # 20 random x values between 0 and 10
y = 2.5 * X + 1.5 + np.random.randn(20) * 2  # True line:  $y = 2.5x + 1.5 + \text{noise}$ 
```

```
# ----- Loss Function (SSE) -----
```

```
def sum_squared_error(w0, w1, X, y):
    y_pred = w0 + w1 * X
    return np.sum((y - y_pred) ** 2)
```

```
# ----- Create Parameter Grid -----
```

```
w0_values = np.linspace(-10, 15, 80)  # range for intercept (w0)
w1_values = np.linspace(-1, 5, 80)     # range for slope (w1)
```

```
W0, W1 = np.meshgrid(w0_values, w1_values)
Loss = np.zeros_like(W0)
```

```
for i in range(len(w1_values)):
    for j in range(len(w0_values)):
        Loss[i, j] = sum_squared_error(W0[i, j], W1[i, j], X, y)
```

```
# ----- Create Parameter Grid -----
w0_values = np.linspace(-10, 15, 80)  # range for intercept (w0)
w1_values = np.linspace(-1, 5, 80)     # range for slope (w1)
```

```
W0, W1 = np.meshgrid(w0_values, w1_values)
Loss = np.zeros_like(W0)
```

```
for i in range(len(w1_values)):
    for j in range(len(w0_values)):
        Loss[i, j] = sum_squared_error(W0[i, j], W1[i, j], X, y)
```

```
# Find the approximate best parameters (minimum loss)
min_idx = np.unravel_index(np.argmin(Loss), Loss.shape)
best_w0 = W0[min_idx]
best_w1 = W1[min_idx]
```

```
print(f"Best parameters found in grid search:")
print(f"w0 (intercept) ≈ {best_w0:.3f}")
print(f"w1 (slope) ≈ {best_w1:.3f}")
```

```
# ----- 3D Plot of Parameter Space -----
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')
```

```
surf = ax.plot_surface(W0, W1, Loss, cmap='viridis', alpha=0.8)
ax.set_xlabel('w0 (intercept)')
ax.set_ylabel('w1 (slope)')
ax.set_zlabel('Sum of Squared Errors (Loss)')
ax.set_title('Parameter Space - Loss Surface for Linear Regression')
```

```
# Mark the minimum point
ax.scatter(best_w0, best_w1, Loss.min(), color='red', s=100, label='Minimum (best parameters)')

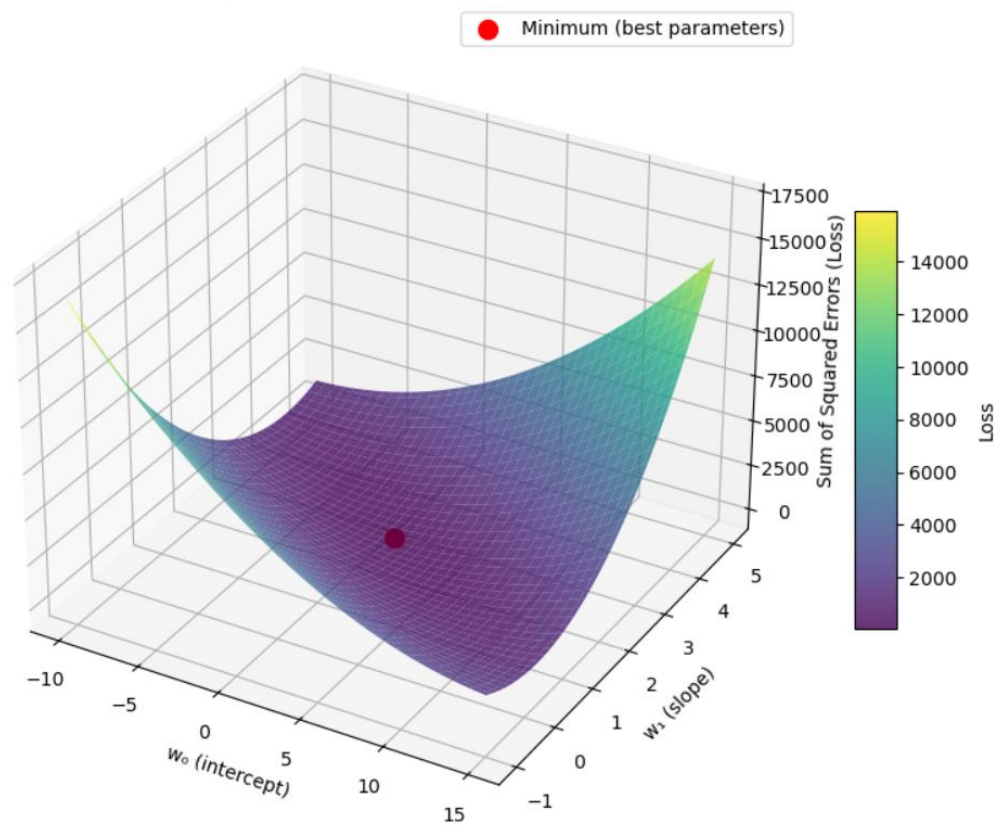
plt.colorbar(surf, ax=ax, shrink=0.5, aspect=10, label='Loss')
ax.legend()
plt.show()
```

```
# ----- Optional: Contour Plot (2D view) -----
```

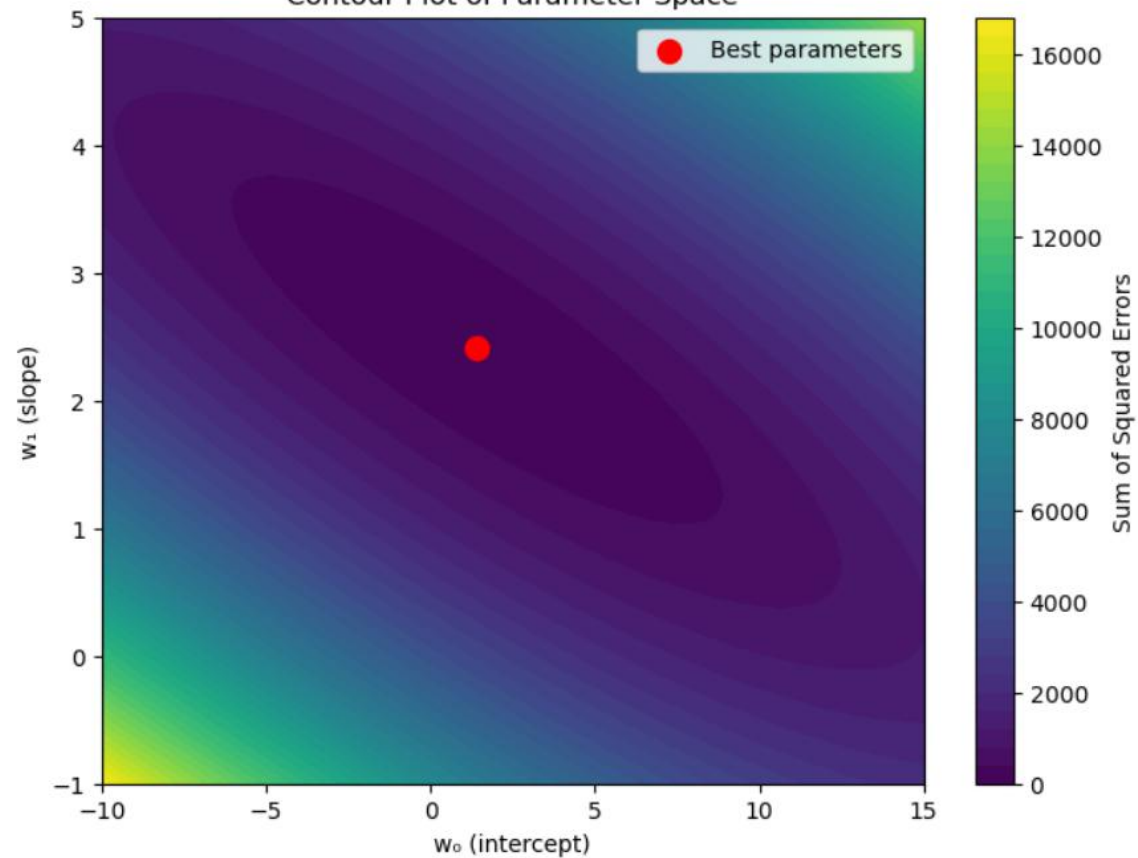
```
plt.figure(figsize=(8, 6))
plt.contourf(W0, W1, Loss, levels=50, cmap='viridis')
plt.colorbar(label='Sum of Squared Errors')
plt.xlabel('w0 (intercept)')
plt.ylabel('w1 (slope)')
plt.title('Contour Plot of Parameter Space')
plt.scatter(best_w0, best_w1, color='red', s=100, label='Best parameters')
plt.legend()
plt.show()
```



Parameter Space - Loss Surface for Linear Regression

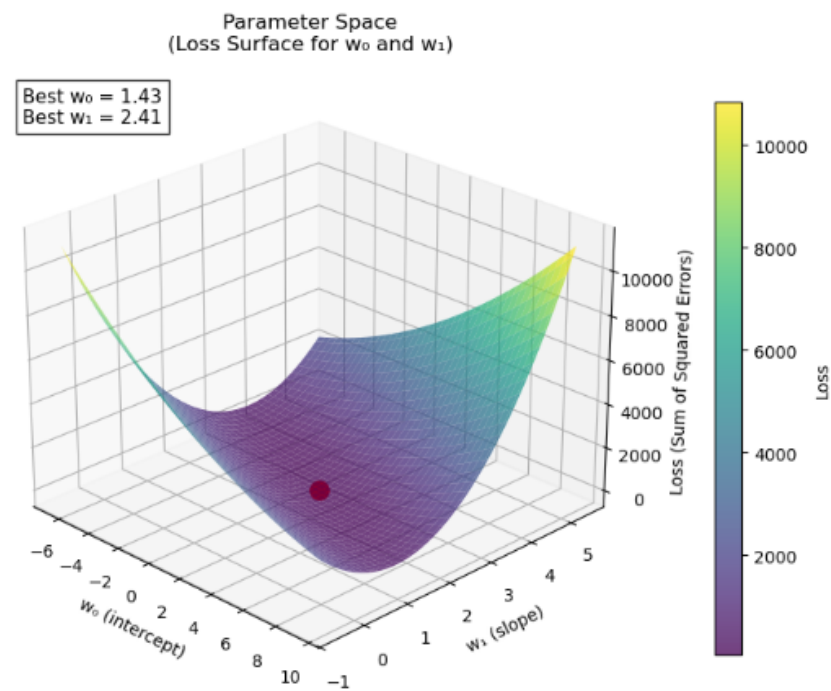
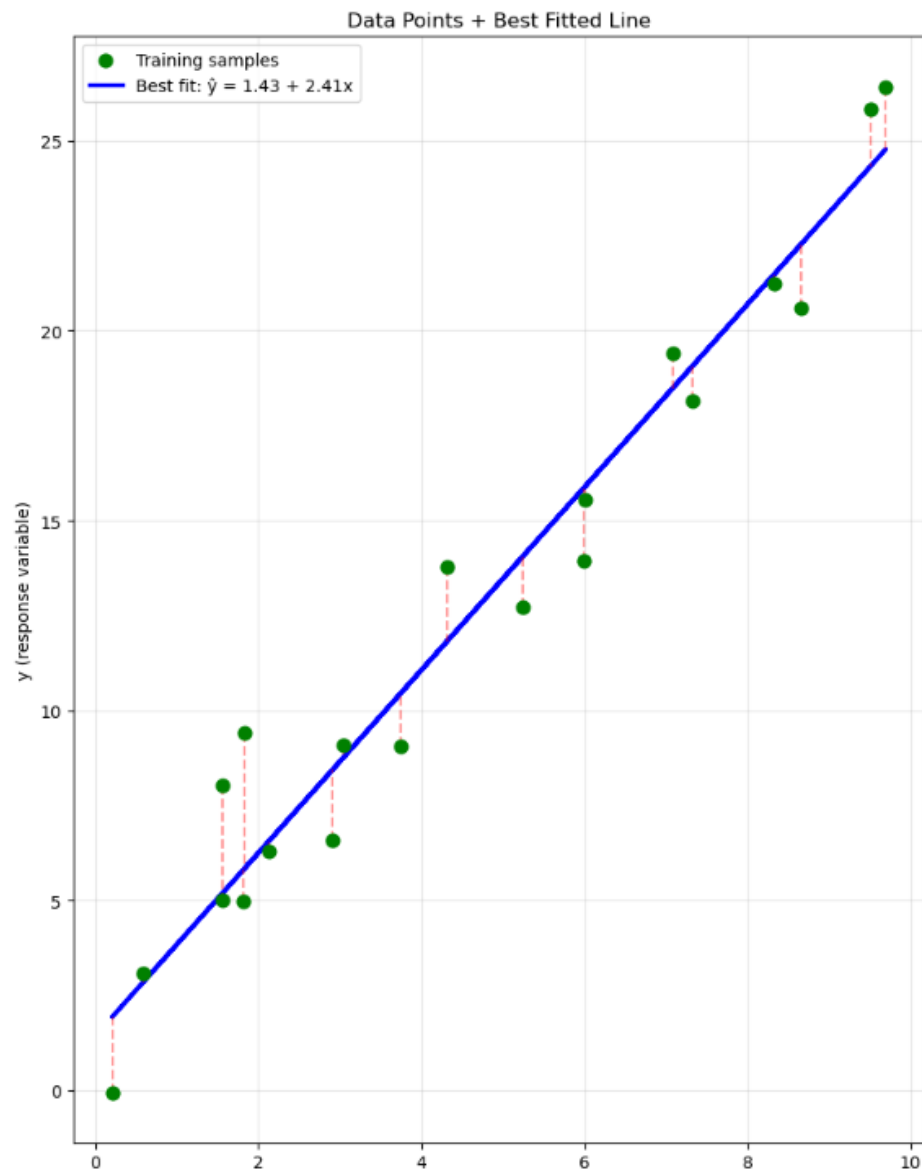


Contour Plot of Parameter Space





Introduction





The simplest deterministic *mathematical* relationship between two variables x and y is a linear relationship: $y = \beta_0 + \beta_1 x$.

The objective of this section is to develop an equivalent *linear probabilistic model*.

If the two (random) variables are probabilistically related, then for a fixed value of x , there is uncertainty in the value of the second variable.

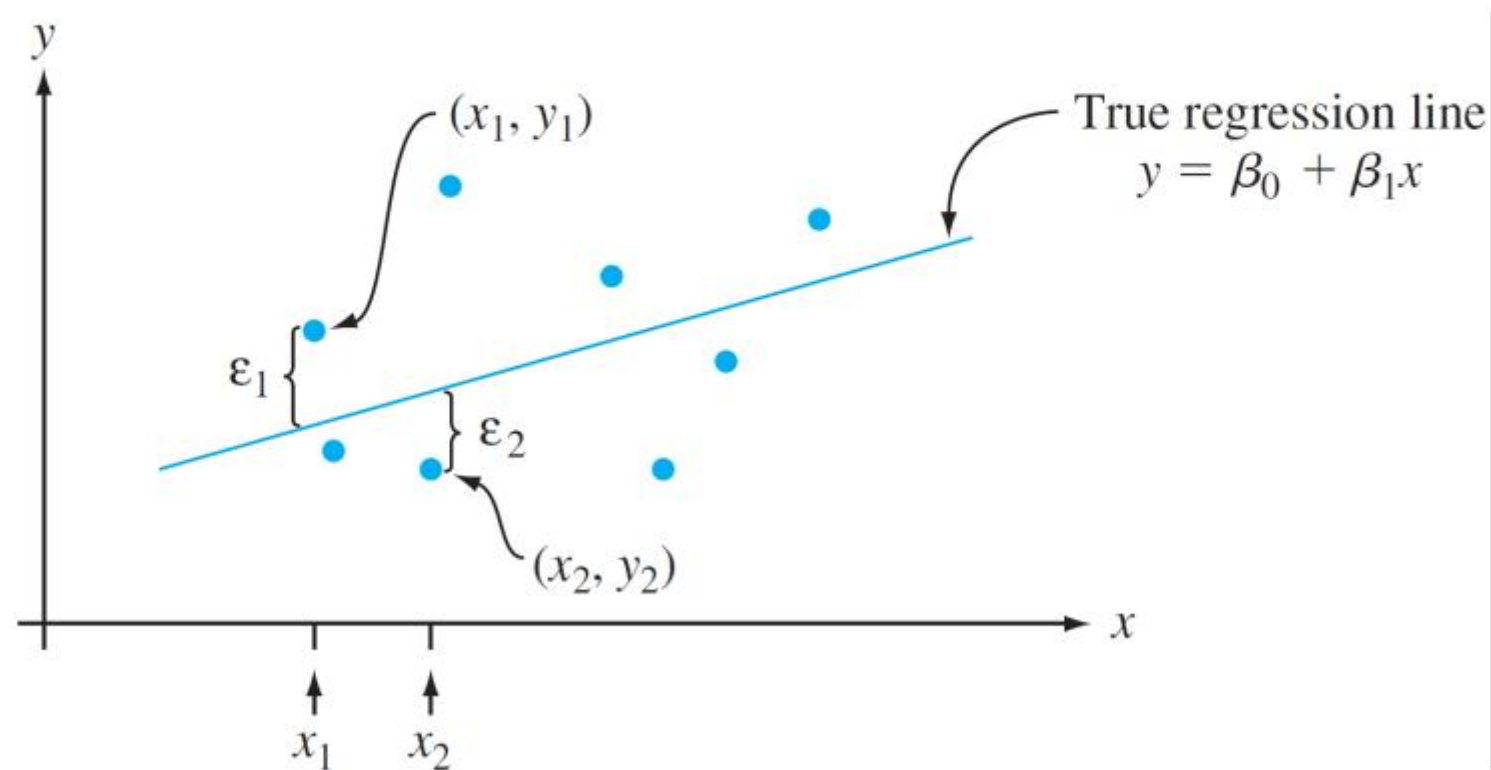
So we assume $Y = \beta_0 + \beta_1 x + \varepsilon$, where ε is a random variable.

2 variables are related linearly “on average” if for fixed x the actual value of Y differs from its expected value by a random amount (*i.e.* there is random error).

-



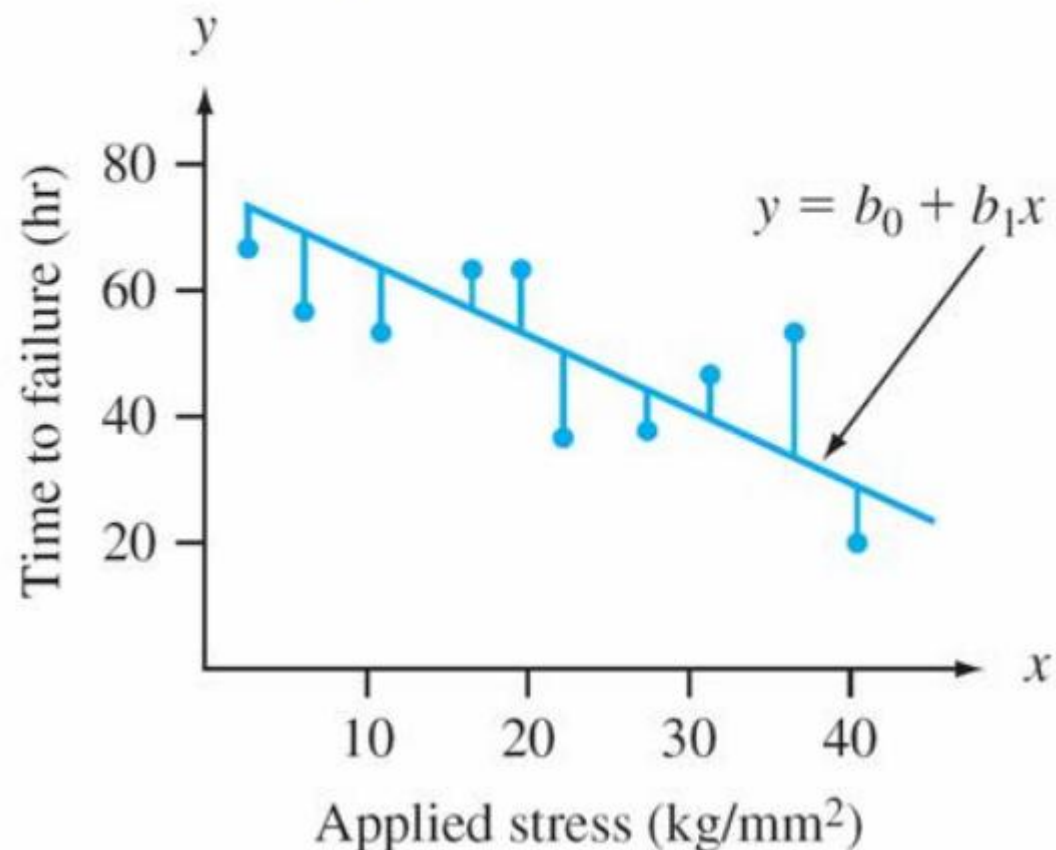
The points $(x_1, y_1), \dots, (x_n, y_n)$ resulting from n independent observations will then be scattered about the true regression line:





The “best fit” line is motivated by the principle of **least squares**, which can be traced back to the German mathematician **Gauss** (1777–1855):

*A line provides the **best fit** to the data if the sum of the squared vertical distances (deviations) from the observed points to that line is as small as it can be.*





The sum of squared vertical deviations from the points $(x_1, y_1), \dots, (x_n, y_n)$ to the line is then

$$f(b_0, b_1) = \sum_{i=1}^n [y_i - (b_0 + b_1 x_i)]^2$$

The point estimates of β_0 and β_1 , denoted by $\hat{\beta}_0$ and $\hat{\beta}_1$, are called the **least squares estimates** – they are those values that minimize $f(b_0, b_1)$.



The least squares estimate of the intercept β_0 of the true regression line is

$$b_0 = \hat{\beta}_0 = \frac{\sum y_i - \hat{\beta}_1 \sum x_i}{n} = \bar{y} - \hat{\beta}_1 \bar{x}$$

The computational formulas for S_{xy} and S_{xx} require only the summary statistics $\sum x_i$, $\sum y_i$, $\sum x_i^2$ and $\sum x_i y_i$.

($\sum y_i^2$ will be needed shortly for the variance.)



• My website simulator ...

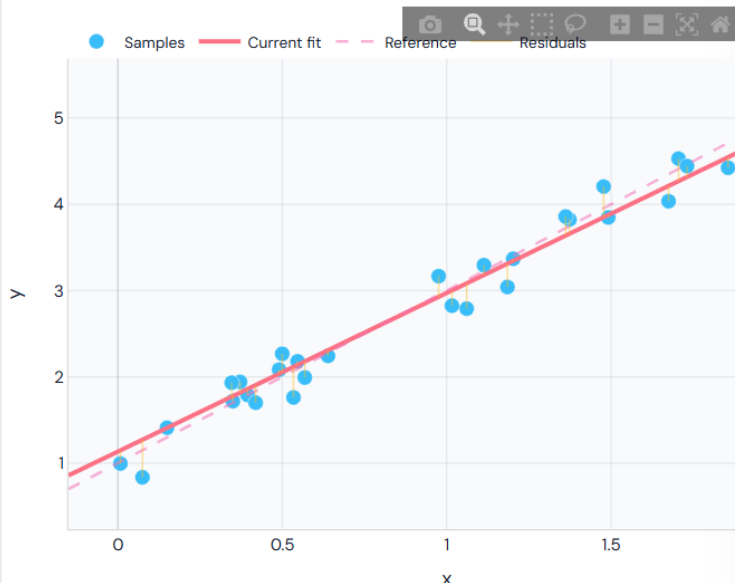
Visualization & Optimization — Sample vs Parameter Space

Linear regression, loss landscape, gradient descent on the loss surface

Animation stopped by user.

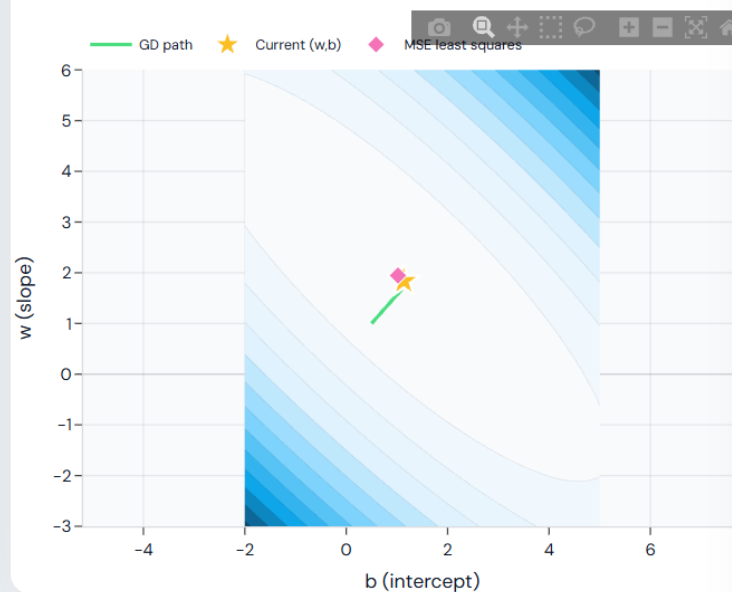
Sample space (data & fit)

Blue: samples; coral: current fit; dashed: generative reference.



Parameter space & loss landscape

Colors: lower loss brighter; star: (w,b); green: GD; diamond: closed-form MSE fit. Click contour.



Prepared by Mohammad Khaloeei

Details

Computation details

Use "Log current state" or run GD to append step-by-step parameters, gradients, and LaTeX formulas.

Log current state

Clear detail log

Step 1 of 40

Parameters $w = 1.0000$, $b = 0.5000$
(before this update)

Data & residuals $n = 28$, $x \in [0.01, 1.86]$, $\overline{x} = 1.3531$, $\overline{y} = 3.1531$

Loss value $\mathcal{L} = 2.136357$

Gradient $g_w = -2.947624$, $g_b = -2.706219$

Update $\eta = 0.1500$, $w' = 1.4421$, $b' = 0.9059$

$$\mathcal{L}_{\text{MSE}}(w, b) = \frac{1}{n} \sum_{i=1}^n (y_i - wx_i - b)^2$$

$$\frac{\partial \mathcal{L}}{\partial w} = -\frac{2}{n} \sum_i x_i r_i, \quad \frac{\partial \mathcal{L}}{\partial b} = -\frac{2}{n} \sum_i r_i, \quad r_i = y_i - wx_i - b$$

$$w \leftarrow w - \eta g_w, \quad b \leftarrow b - \eta g_b$$



Loss function

Mohammad Khalooei

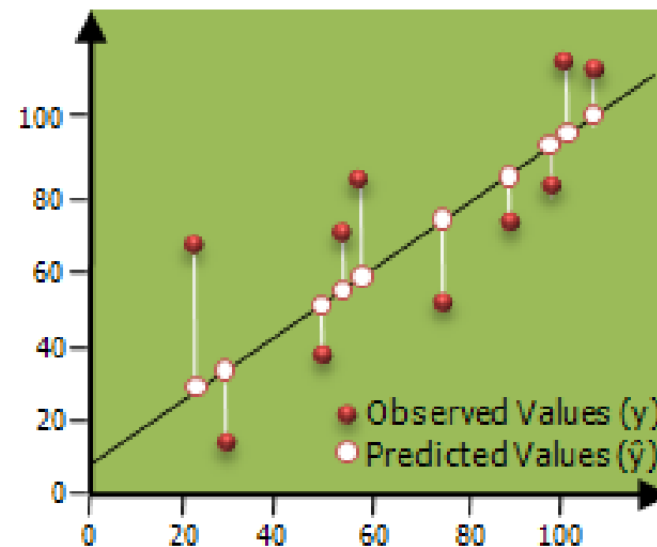




- learn through different methods
- but a fundamental component of the learning process of machine learning algorithms and models is **the loss function**.
- The **loss function** is a mathematical process that quantifies the error margin between a *model's prediction* and the *actual target value*.



- learn through different methods
- but a fundamental component of the learning process of machine learning algorithms and models is **the loss function**.
- The **loss function** is a mathematical process that quantifies the error margin between a *model's prediction* and the *actual target value*.



Observed value=actual target value

Predicted value=model's prediction value



- The loss function is a **measurable way** to gauge the performance and accuracy of a machine learning model.
- In this case, the loss function **acts as a guide** for the learning process within a model or machine learning algorithm!
- The role of the loss function is crucial in the training of machine learning models



- The role of the loss function is crucial in the training of machine learning models and includes the following:
 - **Performance measurement:** Loss functions offer *a clear metric* to evaluate a model's performance by quantifying the difference between predictions and actual results.
 - **Direction for improvement:** Loss functions *guide model* improvement by directing the algorithm to adjust parameters(weights) iteratively to reduce loss and improve predictions.
 - **Balancing bias and variance:** Effective loss functions help balance model bias (*oversimplification*) and variance (*overfitting*), essential for the model's generalization to new data.
 - **Influencing model behavior:** Certain loss functions can affect the model's behavior, such as being *more robust against data outliers* or prioritizing specific types of errors.



- The loss function, also referred to as the **error function**, is a crucial component in machine learning
- that quantifies **the difference between the predicted outputs of a machine learning algorithm and the actual target values.**
- For example, within a regression problem to predict car prices based on historical data, a loss function evaluates a neural network prediction based on a training sample from the training dataset. The loss function quantifies the gap or the error margin of the car price predicted by the network to the actual price.



- The loss function, also referred to as the **error function**, is a crucial component in machine learning
- that quantifies **the difference between the predicted outputs of a machine learning algorithm and the actual target values.**
 - The resulting value, the loss, reflects the accuracy of the model's predictions.



MSE (Mean Squared Error): Average of the squared differences between predicted and actual values. Lower values indicate better fit.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Where y_i is the actual value, $\hat{y}_i$ is the predicted value from the model, and n is the number of data points.

MAE (Mean Absolute Error): Average of the absolute differences between predicted and actual values. More robust to outliers than MSE.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Where y_i is the actual value, $\hat{y}_i$ is the predicted value from the model, and n is the number of data points.

R² (R-squared): Proportion of variance explained by the model, from 0 to 1. Higher values indicate better fit (1 = perfect fit).

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

Where y_i is the actual value, $\hat{y}_i$ is the predicted value, $\bar{y}$ is the mean of actual values, and n is the number of data points.



- The loss function, also referred to as the **error function**, is a crucial component in machine learning
- that quantifies **the difference between the predicted outputs of a machine learning algorithm and the actual target values.**
 - The resulting value, the loss, reflects the accuracy of the model's predictions.

Training models will not be covered in this course

During training, *a learning algorithm* such as the backpropagation algorithm uses the gradient of the loss function with respect to the model's parameters to adjust these parameters and minimize the loss, effectively improving the model's performance on the dataset.

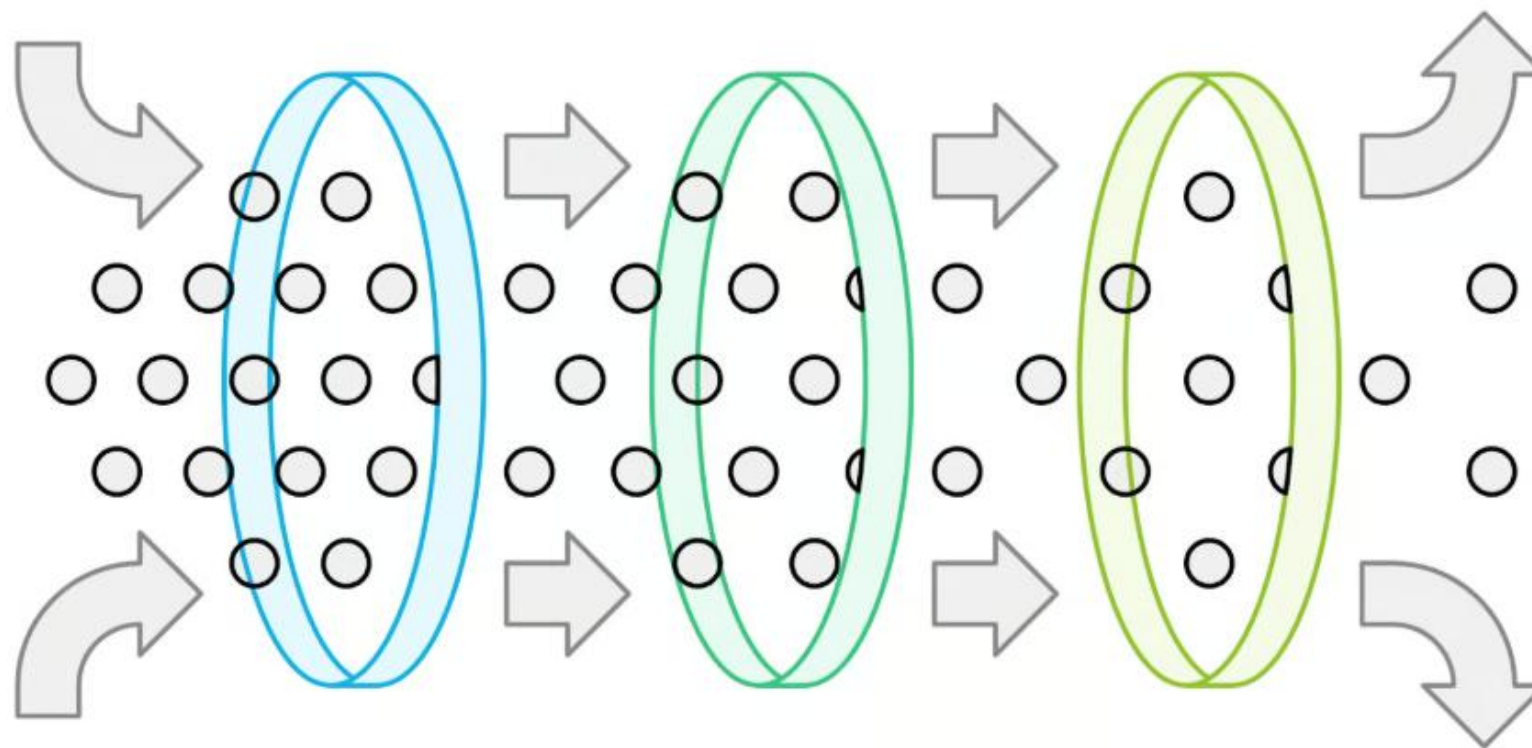


- The loss function, also referred to as the **error function**, is a crucial component in machine learning
- that quantifies **the difference between the predicted outputs of a machine learning algorithm and the actual target values.**
- The resulting value, the loss, reflects the accuracy of the model's predictions.

Training models will not be covered in this course

During training, *a learning algorithm* such as the backpropagation algorithm uses the gradient of the loss function with respect to the model's parameters to adjust these parameters and minimize the loss, effectively improving the model's performance on the dataset.

Instead, Our course focuses on **optimization** and **finding better solutions**,
with particular emphasis on metaheuristic methods.



Calculate Loss

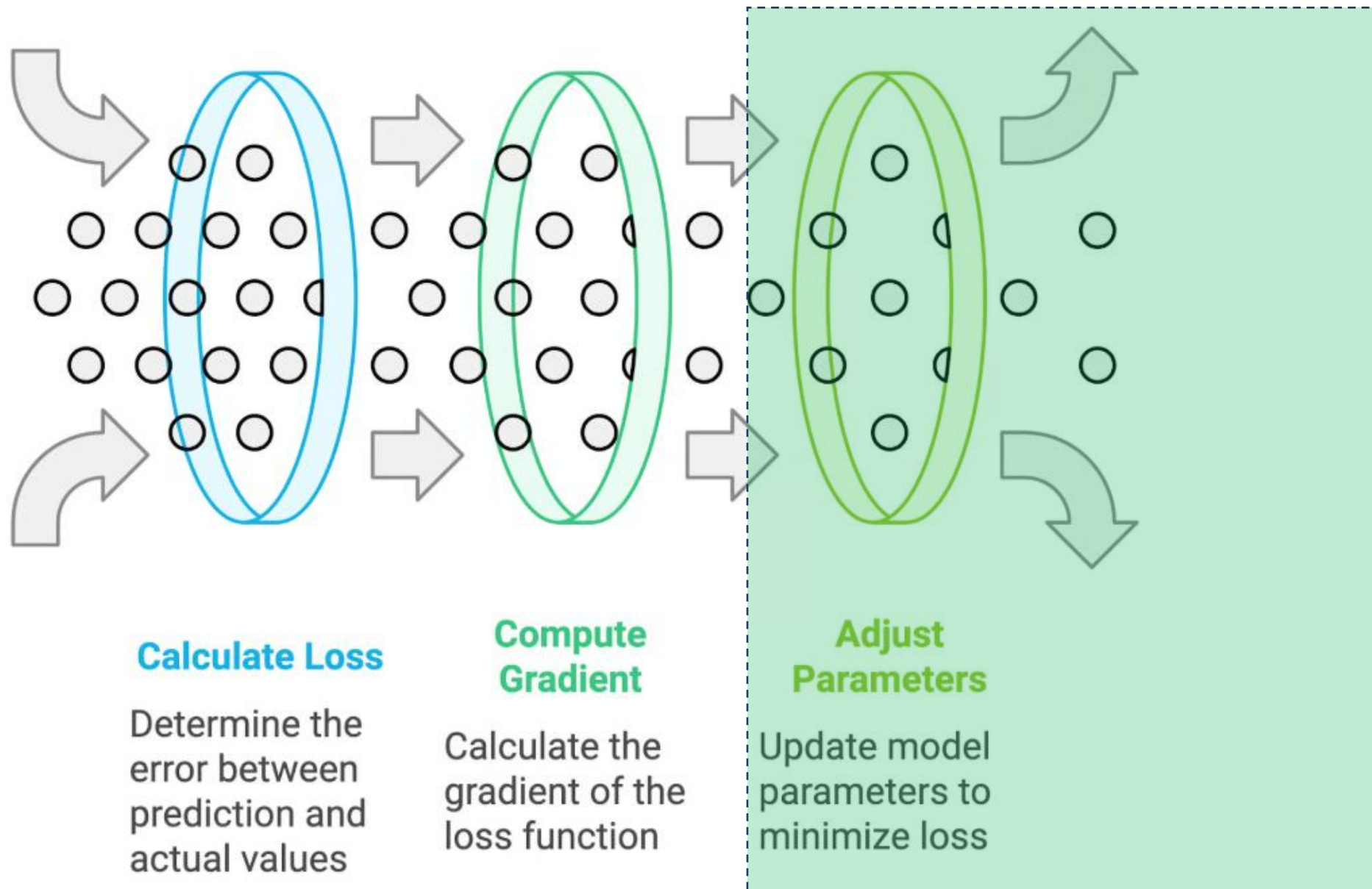
Determine the error between prediction and actual values

Compute Gradient

Calculate the gradient of the loss function

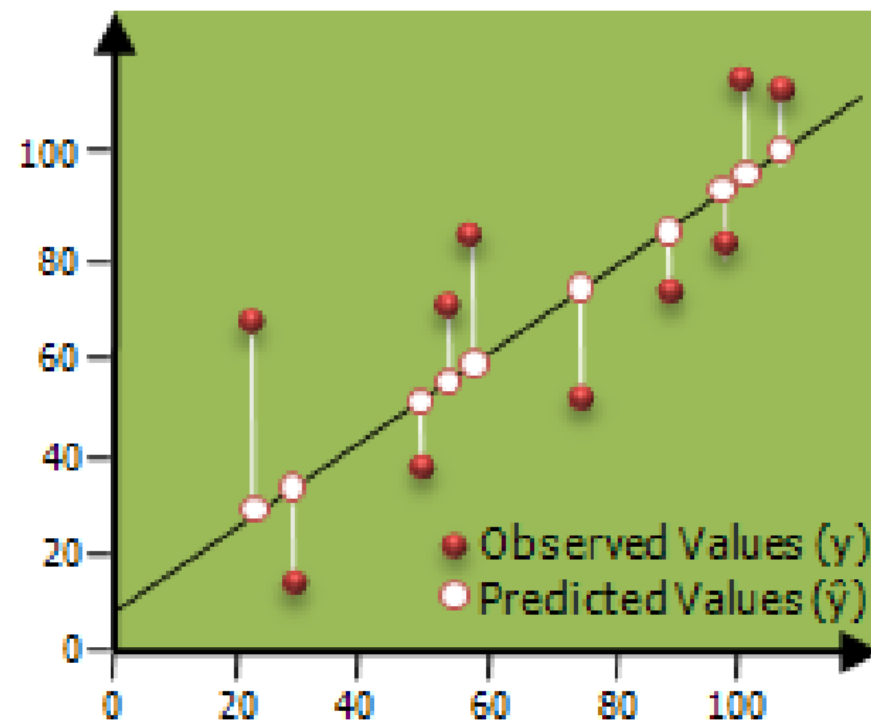
Adjust Parameters

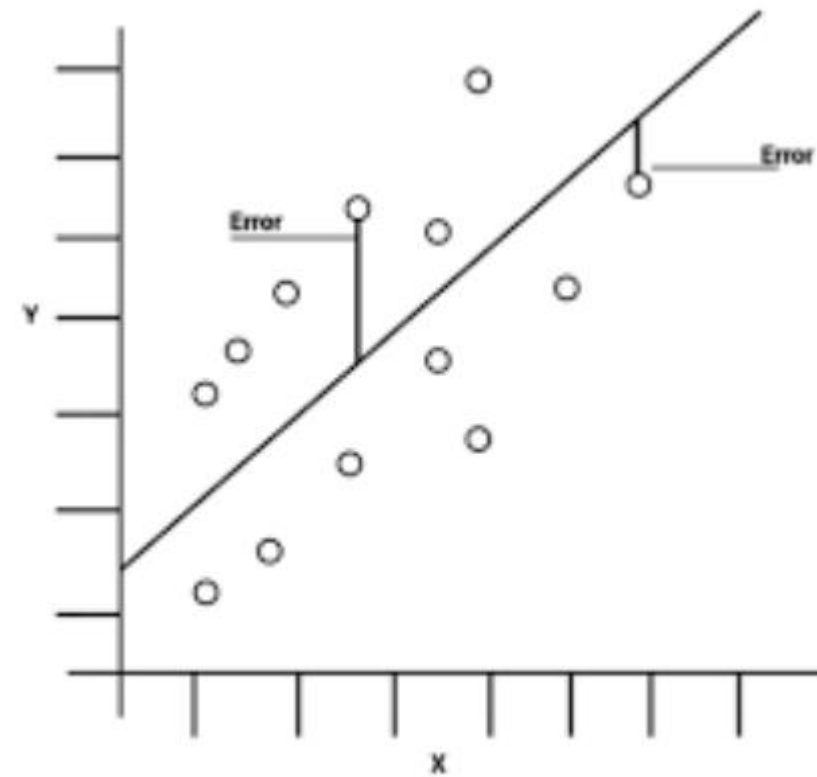
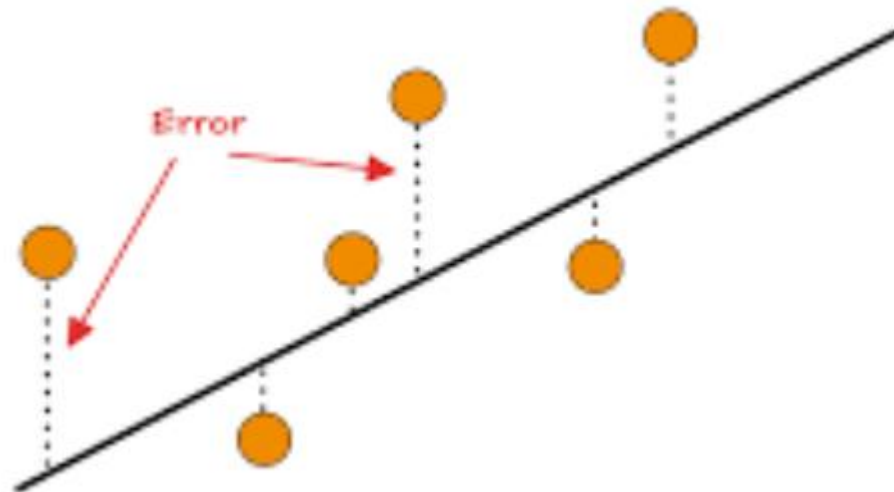
Update model parameters to minimize loss

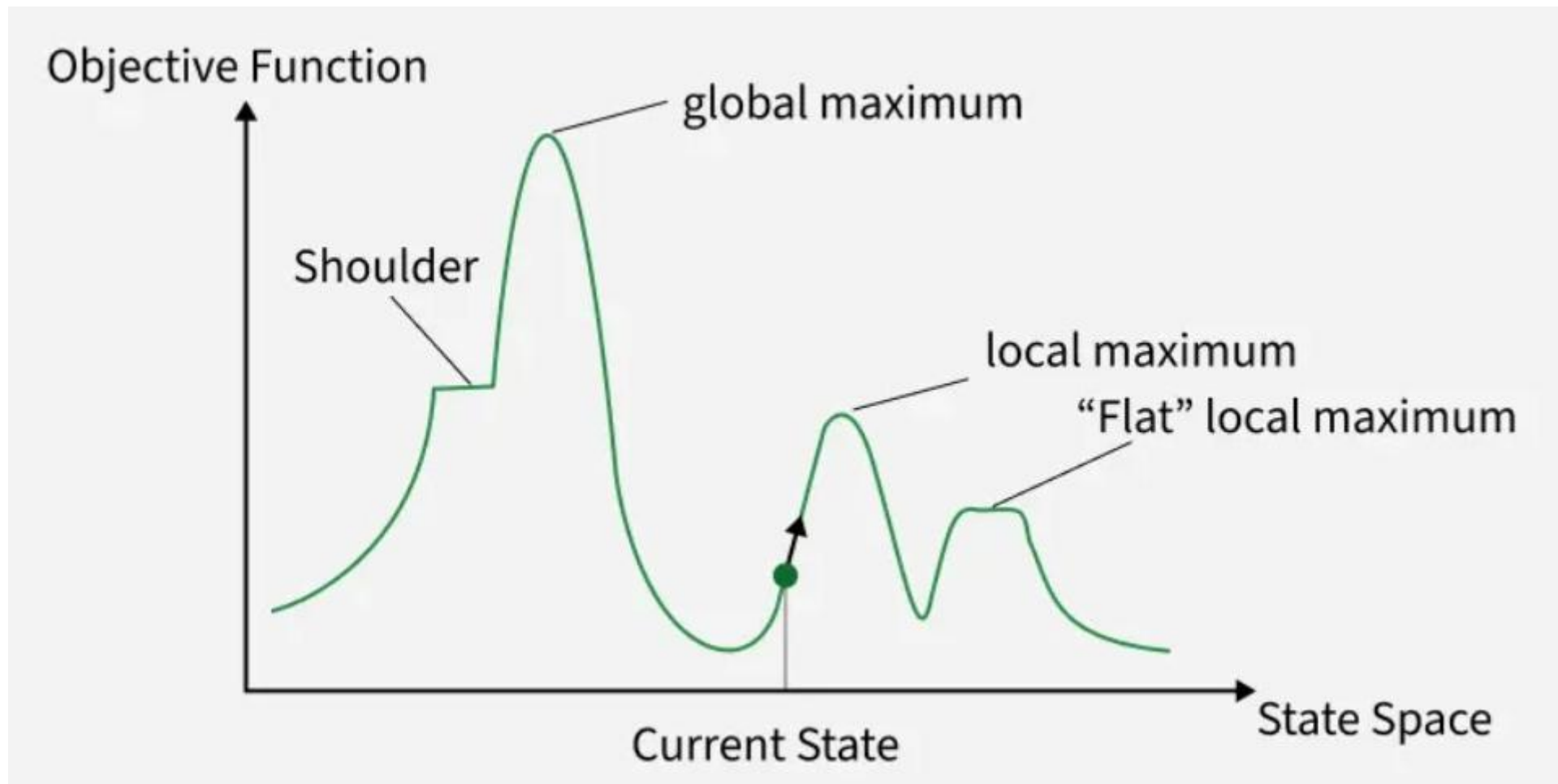


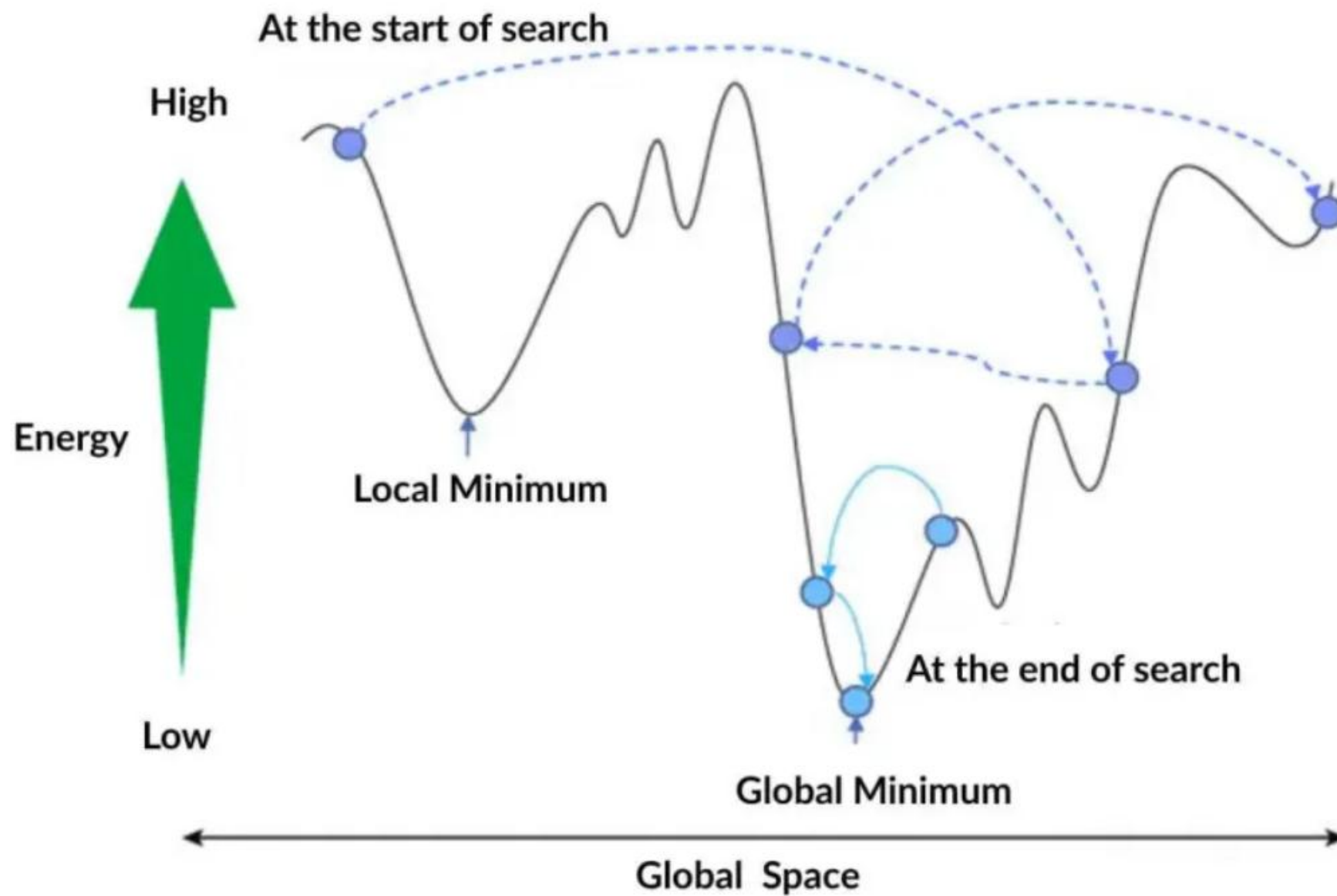


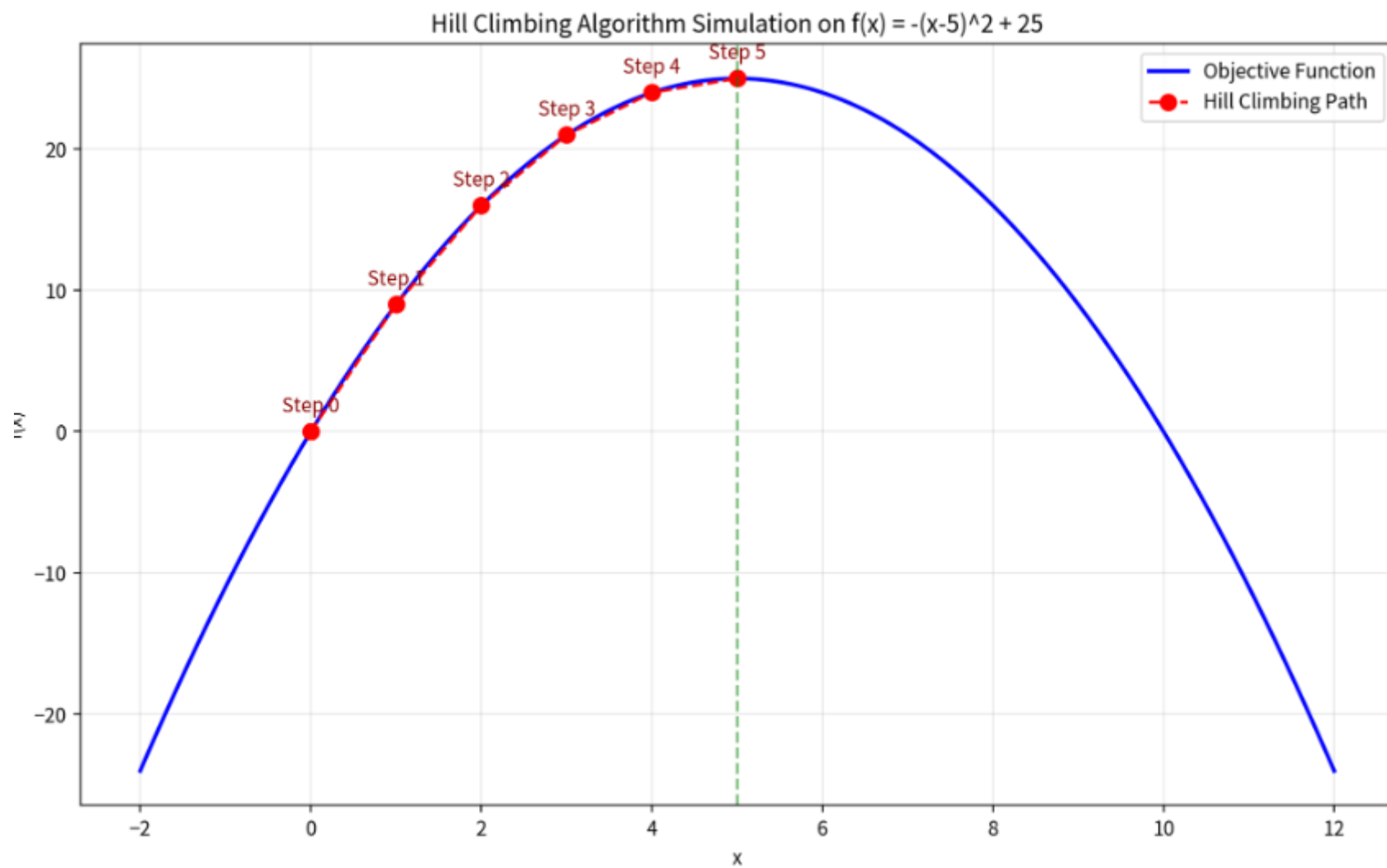
- Ordinary least square







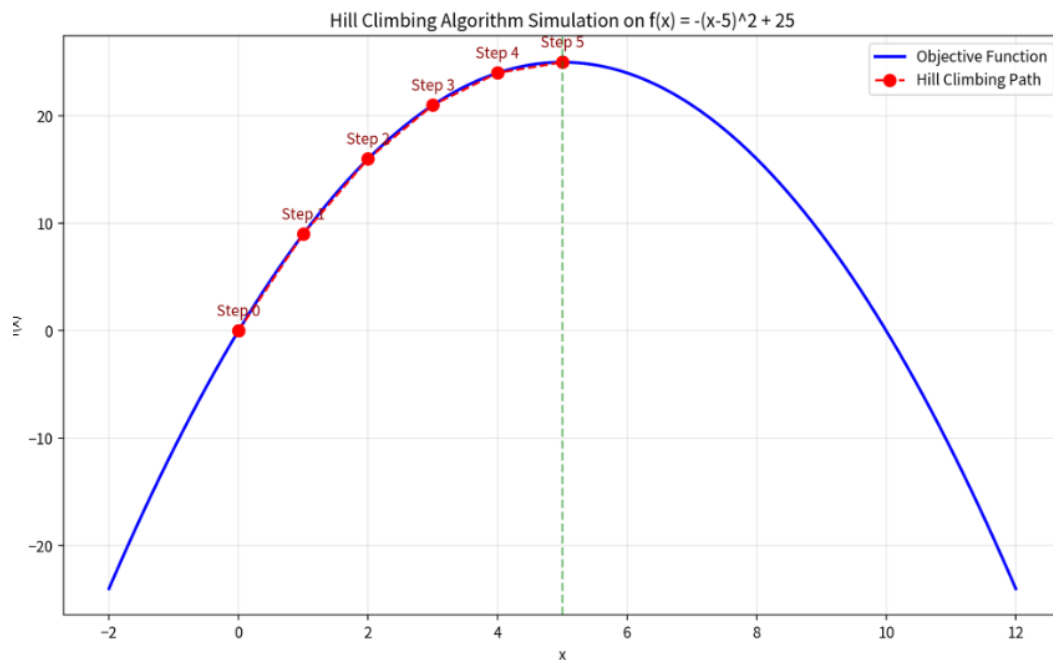






```
import numpy as np
import matplotlib.pyplot as plt
```

```
# Objective function to maximize (example: quadratic with global max at x=5)
def objective(x):
    return -(x - 5)**2 + 25
```



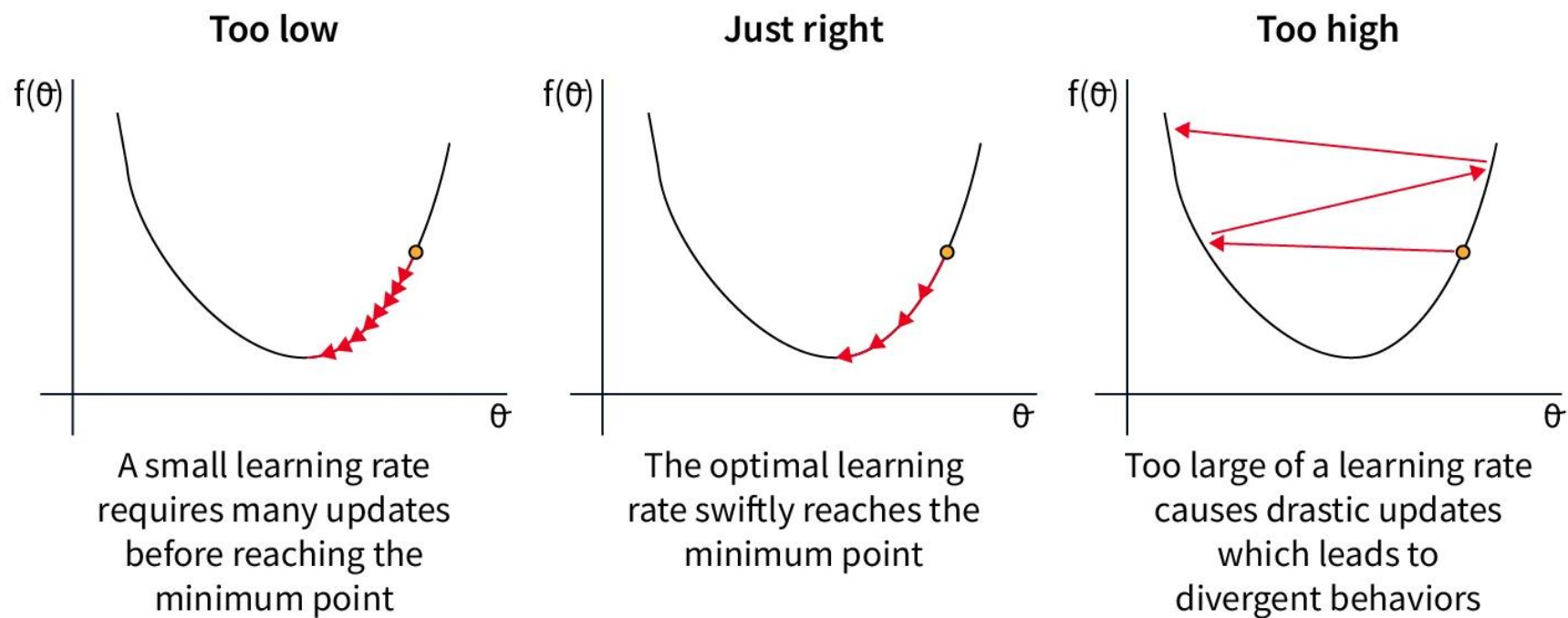
```
# Hill Climbing (steepest ascent in 1D)
def hill_climbing(start_x, step_size=1.0, max_iter=50):
    current_x = start_x
    current_score = objective(current_x)
    path = [(current_x, current_score)]
    print(f"Starting at x = {current_x:.1f}, score = {current_score:.1f}")

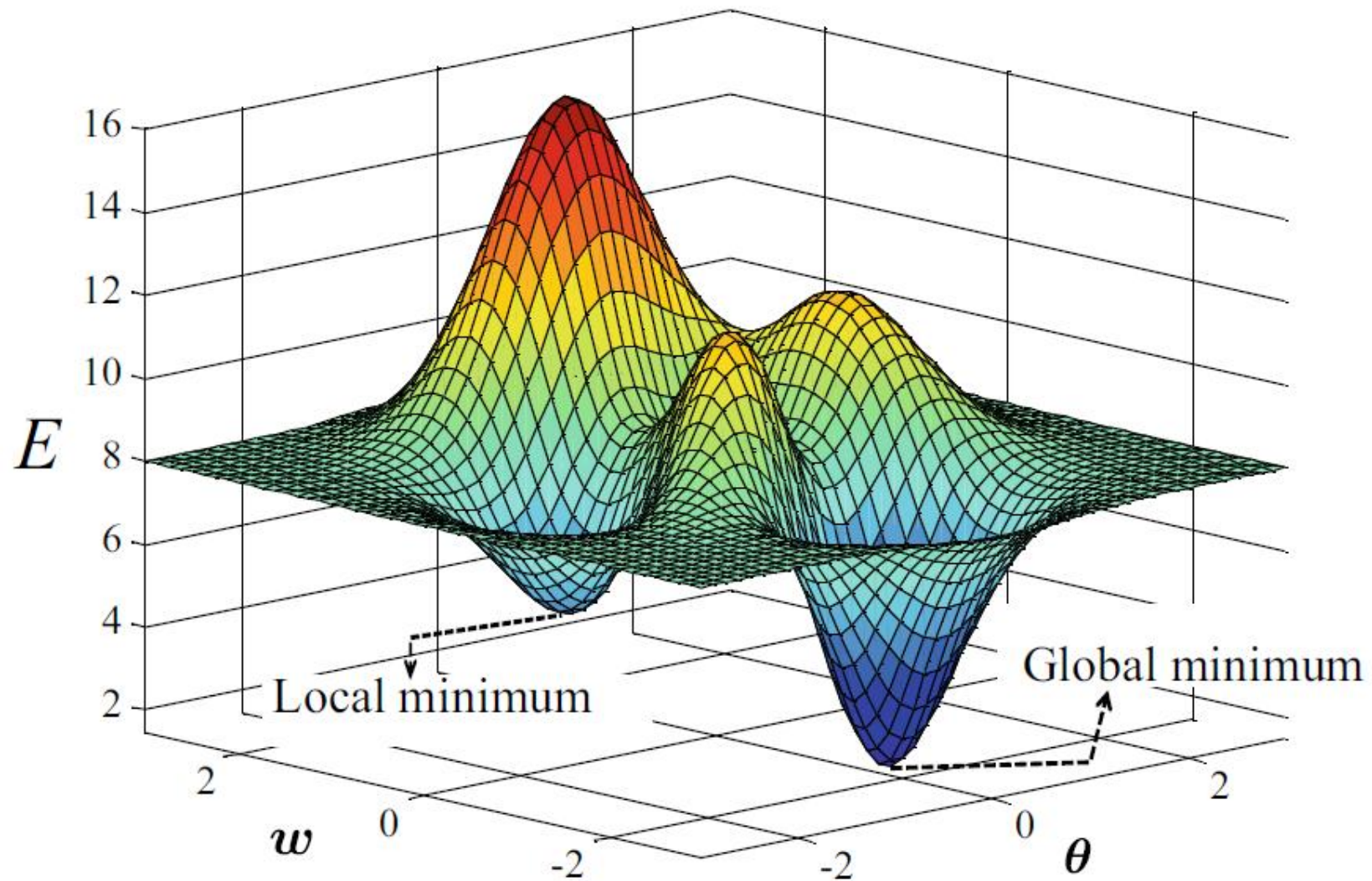
    for step in range(1, max_iter + 1):
        # Generate two neighbors
        left = current_x - step_size
        right = current_x + step_size
        left_score = objective(left)
        right_score = objective(right)

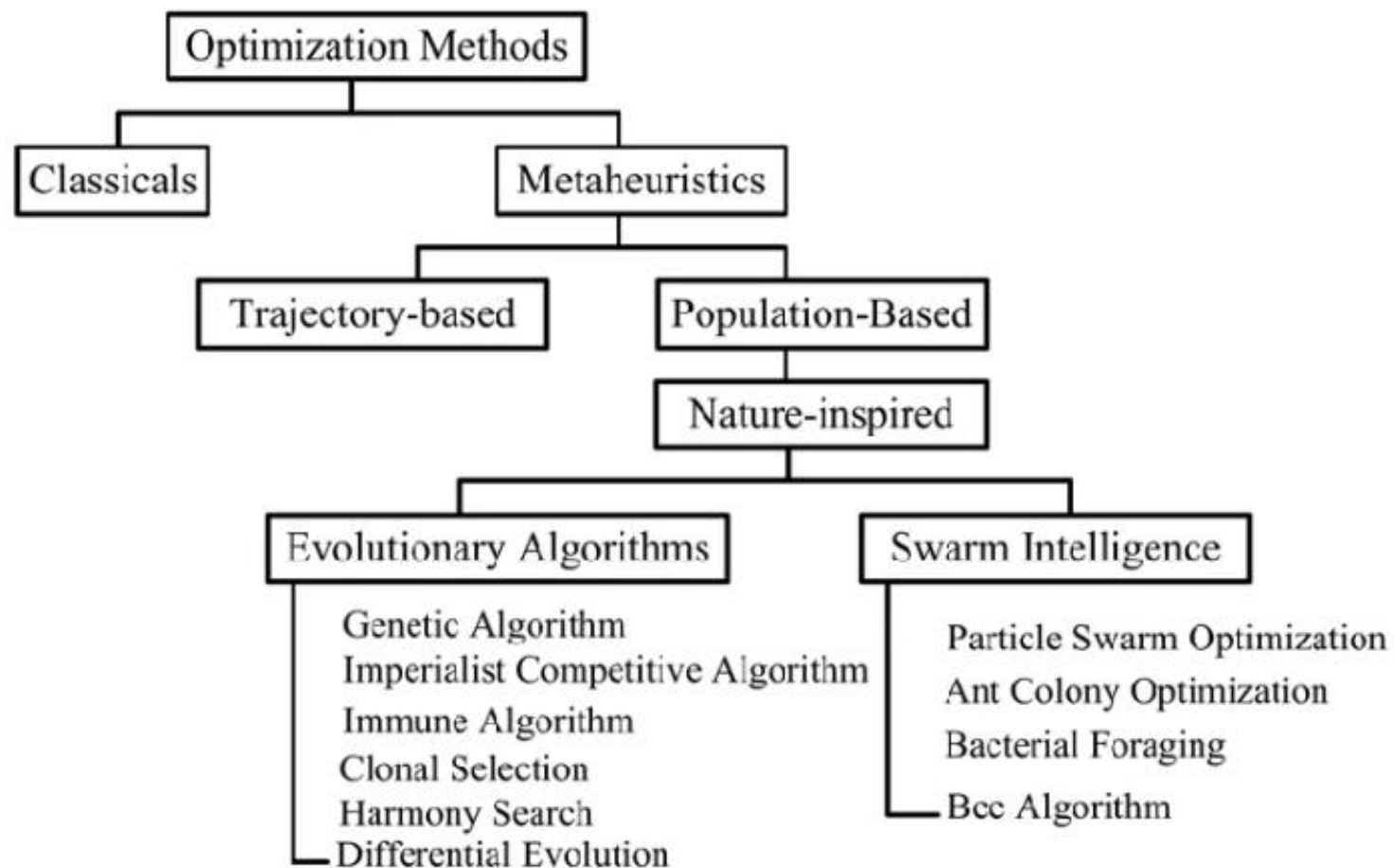
        # Choose the best neighbor if it improves
        if left_score > right_score and left_score > current_score:
            next_x = left
            next_score = left_score
        elif right_score > current_score:
            next_x = right
            next_score = right_score
        else:
            print(f"Step {step}: No improvement. Stopped at local maximum.")
            break

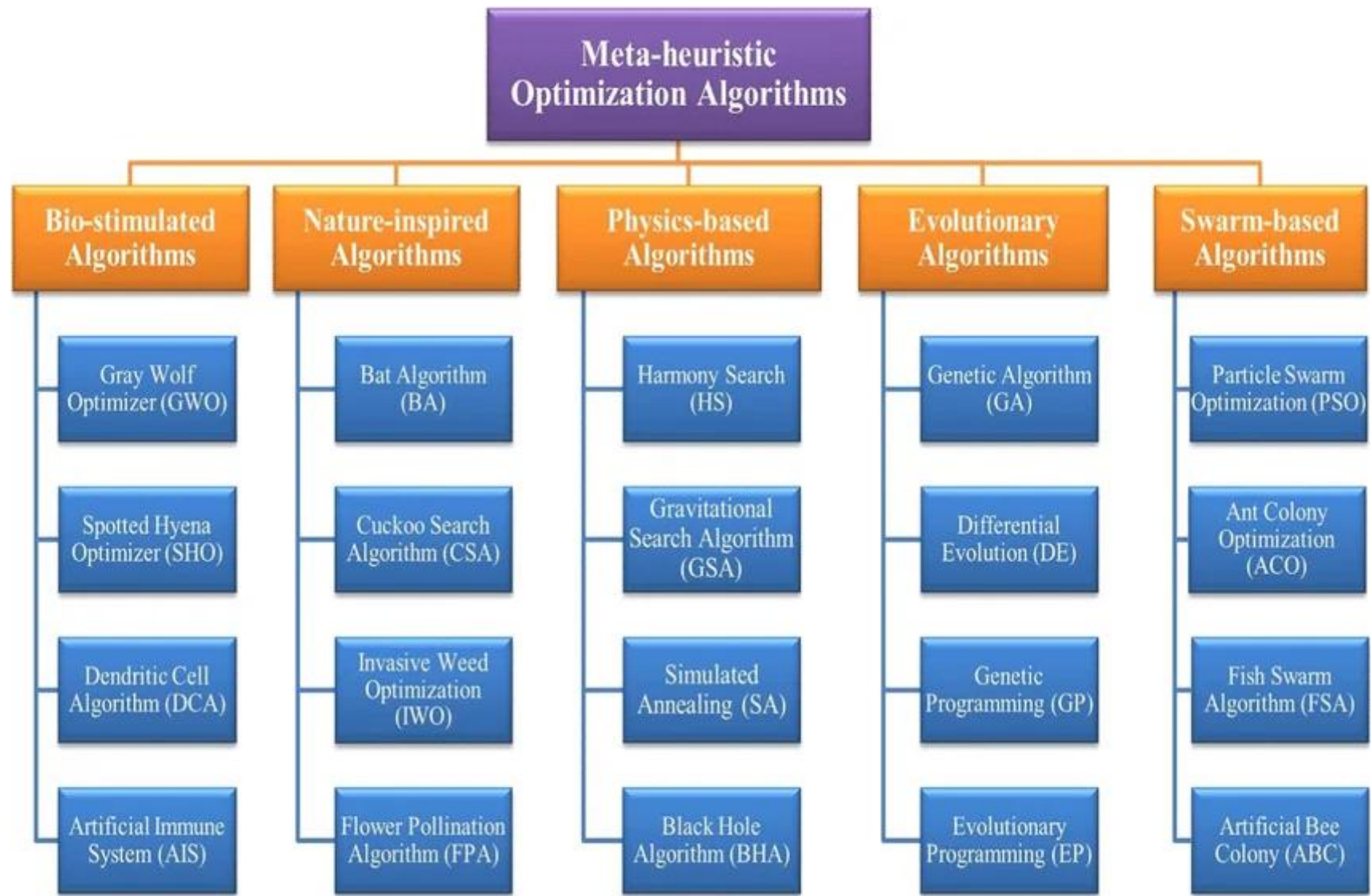
        current_x = next_x
        current_score = next_score
        path.append((current_x, current_score))
        print(f"Step {step}: Moved to x = {current_x:.1f}, score = {current_score:.1f}")

    return path
```









https://www.researchgate.net/publication/333940328_A_comparative_review_of_meta-heuristic_approaches_to_optimize_the_SLA_violation_costs_for_dynamic_execution_of_cloud_services?_tp=eyJjb250ZXh0Ijp7ImZpcnN0UGFnZSI6Ii9kaXJlY3QiLCJwYVWdlIjoX2RpcmVjdCJ9fQ

